IntelliBug: An AI-Assisted System for Bug Severity Classification and Solution Recommendation for Web Applications

by

NURFARHANA NAJIHAH BINTI MOHD FADLI
22003122

Dissertation submitted in partial fulfilment of the requirements for the
Bachelor of Information Technology (Hons)
(Software Quality)

SEPTEMBER 2026

Universiti Teknologi PETRONAS
32610, Bandar Seri Iskandar
Perak Darul Ridzuan

CERTIFICATION OF APPROVAL

IntelliBug: An AI-Assisted System for Bug Severity Classification and Solution Recommendation for Web Applications

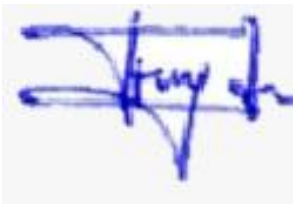
by

NURFARHANA NAJIHAH BINTI MOHD FADLI
22003122

A project dissertation submitted to the Information Technology Programme, Universiti Teknologi PETRONAS, in partial fulfilment of the requirement for the

BACHELOR OF INFORMATION TECHNOLOGY (Hons) (SOFTWARE QUALITY)

Approved by,



_____
(Assoc. Prof Ts. Dr. Toni Anwar)

UNIVERSITI TEKNOLOGI PETRONAS
BANDAR SERI ISKANDAR, PERAK

SEPTEMBER 2026

CERTIFICATION OF ORIGINALITY

This is to certify that I am responsible for the work submitted in this project, that the original work is my own except as specified in the references and acknowledgements, and that the original work contained herein have not been undertaken or done by unspecified sources or persons.



NURFARHANA NAJIHAH BINTI MOHD FADLI

ABSTRACT

Software teams get many bug tickets, and the degree of severity for each one is usually determined manually. Manual determination is slow and dependent upon subjective opinion, delaying the fixing of important bugs. This project developed IntelliBug, a web application that uses artificial intelligence to predict the severity level of a bug report. To achieve the results, an XGBoost classifier was trained using around 478,000 bug records captured from the Eclipse Bugzilla database. The titles of the reports were cleaned, tokenised and converted into 10,000 TF-IDF features, and balanced sample weights were applied because Medium reports form almost 89% of the dataset. A Flask web application with Google sign-in, a MySQL database, a dashboard and an analytics page was built around the model. A large language model, Claude, generates a structured recommendation made of a root cause, a suggested fix, and the best practice for every report. On a held-out test set of 95,731 reports, the deployed model reached an accuracy of 55.5% and a macro F1-score of 0.365, with a macro recall of 52.6%. It found about half of the Critical and Low reports, while a majority-class baseline found none. Twelve automated Selenium tests were written for the web application, and each test passed. Five individuals managed to carry out each of the four usability assignments successfully and the average SUS result was 69.0, which is higher than the average value of 68. The recommendations were rated between 4.14 and 4.34 out of 5 by five raters in terms of relevance, correctness, and actionability. The findings show severity prediction from short text to be possible but not precise for the minority classes. IntelliBug therefore works as a triage assistant whose output a tester confirms. The project is important because it joins prediction, repair advice and defect tracking in one evaluated tool. Further developments will include training on complete bug reports, testing transformer-based models and evaluating recommendations with practitioners.

ACKNOWLEDGEMENTS

I wish to express deep gratitude to the main supervisor, Assoc. Prof. Ts. Dr. Toni Anwar. The help, support, and especially useful evaluation from him helped me with every step of the project IntelliBug during two semesters of work on my Final Year Project. Thanks to him, I received good comments and recommendations on the project, and his trust helped me to continue my work on it.

My sincere gratitude goes to the coordinators and committee members of the Final Year Project subject in the Department of Computing at the Universiti Teknologi PETRONAS. With the help of their suggestions, recommendations, and evaluations of the subject, this paper has been well established and organized.

I am also grateful to my friends who agreed to devote their time to be participants in the usability testing and make ratings for my recommendations. Their honest evaluations made it possible to know what works well in IntelliBug and what needs improvement. Their ideas which include a step-by-step solution and seriousness scoring will be described in Chapter 5.

I owe a debt of gratitude to the open-source community. The Eclipse Bugzilla data, Flask, XGBoost, scikit-learn and Selenium are maintained by people who share their work freely, and they made it possible to build and test a complete system within the time of one project. Claude API gave the recommendation module the language ability to turn a bug report into a clear root cause, fix, and best practice.

Thanks also go to the fellow students of the programme, who shared ideas, advice, and encouragement during the semester.

Finally, I would like to thank my family for their unwavering support throughout this project. I would like to thank them for their endless encouragement, patience, and prayers throughout the busy coding, writing, and revising weeks.

TABLE OF CONTENTS

CERTIFICATION OF APPROVAL ..... **i**

CERTIFICATION OF ORIGINALITY ..... **ii**

ABSTRACT ..... **iii**

ACKNOWLEDGEMENTS ..... **iv**

CHAPTER 1: INTRODUCTION **1**

1.1 Background of Study 1

1.2 Problem Statement 2

1.3 Research Questions 2

1.4 Aim of the Study 3

1.5 Research Objectives 3

1.6 Scope of Study 3

1.7 Significance of Research 4

1.8 Thesis Structure 4

CHAPTER 2: LITERATURE REVIEW AND THEORY **6**

2.1 Introduction 6

2.2 Theoretical Framework 6

2.3 Related Works 8

2.4 Research Gap 12

2.5 Chapter Summary 13

CHAPTER 3: METHODOLOGY AND PROJECT WORK **14**

3.1 Introduction 14

3.2 Dataset 15

3.3 Text Preprocessing and Feature Extraction 15

3.4 Model Development 16

3.5 System Design 17

3.6 System Implementation 22

3.7 Testing and Evaluation Plan 22

3.8 Tools and Technologies 24

3.9 Project Timeline and Key Milestones 25

3.10 Chapter Summary 26

CHAPTER 4: RESULTS AND DISCUSSION **27**

4.1 Introduction 27

4.2 Data Characteristics 27

4.3 Model Performance 28

4.4 System Implementation Results 30

4.5 Functional Testing Results 32
4.6 Recommendation Quality 34
4.7 Usability Evaluation Results 35
4.8 Discussion 37
4.9 Chapter Summary 41
CHAPTER 5: CONCLUSION AND RECOMMENDATION 42
5.1 Conclusion 42
5.2 Recommendation for Future Work 43
REFERENCES 45
APPENDICES 49
Appendix A. Selenium Test Suite (test_ui.py) 49
Appendix B. System Usability Scale Questionnaire 52
Appendix C. Project Timeline (FYP II) 57
Appendix D. Tuning Experiment Output 58
Appendix E. Source Code Repository.. 58

LIST OF FIGURES

Figure 3.1: Project methodology and phases 14
Figure 3.2: Machine learning pipeline 16
Figure 3.3: System architecture of IntelliBug 18
Figure 3.4: Use case diagram of IntelliBug 19
Figure 3.5: Sequence diagram of the bug submission 20
Figure 3.6: Entity-relationship diagram of the database 20
Figure 4.1: Class distribution of the test set 27
Figure 4.2: Confusion matrix on the test set 28
Figure 4.3: Precision, recall, and F1-score per class 29
Figure 4.4: Login page with Google authenticator 30
Figure 4.5: Dashboard page with recent bug reports and screenshots 31
Figure 4.6: Submit Bug Reports page with some required fields to fill 31
Figure 4.7: Bug Reports page with search and filters 31
Figure 4.8: Bug Analytics page with the bug trend, open bugs by product and model performance 32
Figure 4.9: Settings page with account details and appearance options 32
Figure 4.10: Mean rating of the recommendations with standard deviation 34
Figure 4.11: SUS score of each participant against the benchmark of 68 35
Figure A.1: Final run of the Selenium suite in the pytest-html report, 19 October 2026 49
Figure B.1: Responses to SUS statement 1 52
Figure B.2: Responses to SUS statement 2 53
Figure B.3: Responses to SUS statement 3 53
Figure B.4: Responses to SUS statement 4 53
Figure B.5: Responses to SUS statement 5 54
Figure B.6: Responses to SUS statement 6 54
Figure B.7: Responses to SUS statement 7 54
Figure B.8: Responses to SUS statement 8 55
Figure B.9: Responses to SUS statement 9 55
Figure B.10: Responses to SUS statement 10 55
Figure B.11: Answers to open question 1, what did you like most about IntelliBug 56
Figure B.12: Answers to open question 2, what was confusing or hard to use 56
Figure C.1: Gantt chart of the FYP II activities over twelve weeks 57

Figure D.1: Terminal output of the run with 5,000 TF-IDF terms 58
Figure D.2: Terminal output of the run with 10,000 TF-IDF terms 58

LIST OF TABLES

Table 1.1: Alignment of the problem, the objectives and the evidence 3
Table 2.1: Comparison of related studies and the proposed system 11
Table 3.1: Project phases and deliverables 14
Table 3.2: Severity mapping used in this project 15
Table 3.3: XGBoost hyperparameters 17
Table 3.4: Fields of the users table 21
Table 3.5: Fields of the bug_reports table 21
Table 3.6: Pages and functions of IntelliBug 22
Table 3.7: Usability test tasks 23
Table 3.8: Tools and technologies 24
Table 3.9: Key milestones of FYP II 26
Table 4.1: Number of records after each preparation step 27
Table 4.2: Overall performance of the deployed model 28
Table 4.3: Per-class results computed from the confusion matrix 29
Table 4.4: Comparison with a majority-class baseline 29
Table 4.5: Tuning experiments 30
Table 4.6: Selenium test cases and results 32
Table 4.7: Defects found during testing 33
Table 4.8: Mean rating of the recommendations (1 to 5) 34
Table 4.9: Task completion rate 35
Table 4.10: SUS scores of the participants 35
Table 4.11: Mean item contribution to the SUS score (maximum 4) 36
Table 4.12: Achievement of the research objectives 37
Table 4.13: Reported results of related studies and IntelliBug 39
Table B.1: SUS statements adapted for IntelliBug 52
Table C.1: FYP II activities 57

ABBREVIATIONS AND NOMENCLATURES

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CRISP-DM	Cross-Industry Standard Process for Data Mining
FYP	Final Year Project
F1	Harmonic mean of precision and recall
HTML	HyperText Markup Language
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
OAuth	Open Authorisation
ORM	Object-Relational Mapping
QA	Quality Assurance
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SUS	System Usability Scale
TF	Term Frequency
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
UTP	Universiti Teknologi PETRONAS
XGBoost	eXtreme Gradient Boosting
TP, FP, FN	True positives, false positives, false negatives for one class
K	Number of severity classes (K = 3)
N	Number of documents in the corpus
df(t)	Number of documents which contain term t

CHAPTER 1
INTRODUCTION

1.1 Background of Study

Software is fundamental to many aspects of life including banking, education, healthcare, and public services. As web applications become larger and more frequent, the number of bugs identified during testing also increases. Bug tracking systems such as Bugzilla and Jira are used to track these bugs. Each bug is represented by a bug report. A bug report will include title, description, the product, and environment that the bug has occurred on. [1]

Each report will have a severity attached. This indicates how serious the issue is to the system and users. Bugzilla has seven categories of severity: blocker, critical, major, normal, minor, trivial and enhancement [1]. Severity helps inform the order in which bugs are fixed. If a high severity bug takes time to fix it will affect users and release dates. Most open-source projects have severity added by the person reporting the issue and reviewed by a triage. This means that severity is based on opinion. [2]

Machine learning and NLP techniques have been used to help with this problem. A classifier is trained to understand the relationship between words in previous reports and severity. It can then predict the severity of a new report. [3] Gradient boosting techniques such as XGBoost have had impressive results using structured and text-based features. [4] Using large language models (LLMs) means that the model can also generate human readable explanations and solutions based on a brief description of the failure. [5]

Most studies only focus on classifying the severity. The tester will still need to read the results and decide on a solution to the bug. Defect tracking must also be done in another tool. This project uses severity classification, solution recommendations, and defect tracking in one web application called IntelliBug. This means that the QA team can go from a raw report to an actionable report.

1.2 Problem Statement

Manual classification of bugs can be a lengthy and subjective process. This can lead to inconsistencies between different reporters and cause delays to fixing critical bugs due to unclear classifications [6]. A busy project will also result in a larger volume of reports than can be easily accessed by the triage team.

There are automated solutions available, however most are research prototypes which have been evaluated offline [7]. These only provide a label with no guidance on what caused the issue and how to fix it. There is also no dashboard provided to track the report.

Another problem is class imbalance. In the Eclipse Bugzilla dataset used within this project there are approximately 89% of reports in the medium category. If a model always predicted Medium then there would be a high accuracy but none of the Critical or Low reports would be detected [8]. It is therefore important that the metrics used are fair and include macro averaged metrics.

The issue that this project aims to solve is there is no integrated system which can predict the severity of bugs, recommend solutions, and allow tracking of defects within web applications.

1.3 Research Questions

The study seeks to answer the following research questions:

i. How is the severity of a bug report classified automatically from its text using TF-IDF features and an XGBoost classifier?

ii. How is a useful solution generated for each classified bug using a large language model?

iii. How does a web dashboard with automated classification and recommendation support defect management tasks?

iv. To what extent are the accuracy, performance, and usability of the proposed system acceptable?

1.4 Aim of the Study

The aim of this study is to develop and evaluate IntelliBug, an AI-assisted web system which predicts the severity of software bug reports and recommends solutions, to improve software testing and defect management.

1.5 Research Objectives

The objectives of this project are:

i. To develop an AI-based system for bug severity classification.

ii. To provide automated solution recommendations for detected software bugs.

iii. To improve software testing and defect management efficiency.

iv. To evaluate the accuracy and performance of the proposed system.

Objectives i to iv answer research questions i to iv, respectively. Table 1.1 shows how each objective answers one part of the problem statement and how the result is measured.

Table 1.1: Alignment of the problem, the objectives and the evidence

Problem	Objective	Evidence in this study
Manual severity assignment is slow and subjective.	i. Develop an AI-based system for bug severity classification.	XGBoost model in the system; macro F1-score and recall per class (Section 4.3).
Existing approaches return a label with no repair guidance.	ii. Provide automated solution recommendations.	Claude recommendation module; ratings of relevance, correctness and actionability (Section 4.6).
No integrated tool predicts, advises and tracks a defect.	iii. Improve software testing and defect management efficiency.	Dashboard, filters, status tracking and analytics; task completion rate (Sections 4.4 and 4.7).
Accuracy misleads on imbalanced data, and tools are rarely evaluated with users.	iv. Evaluate the accuracy and performance of the system.	Macro-averaged metrics, majority-class baseline, 12 Selenium tests and SUS (Sections 4.3, 4.5 and 4.7).

1.6 Scope of Study

The study covers the following:

- Severity prediction in three groups, namely Critical, Medium, and Low, derived from the six Bugzilla severity values.

- Model training on the public Eclipse Bugzilla dataset (July 2022 export) using report titles as text input.
- A web application for bug submission, tracking, filtering, and analytics, with Google sign-in and per-user data access.
- Solution recommendation through the Claude API, returned as a root cause, a suggested fix and best practice.
- Functional testing with Selenium and usability testing with the System Usability Scale (SUS).

The study excludes duplicate-bug detection, automatic developer assignment, priority prediction, source-code level fault localisation, non-English reports, and public deployment on a production server. The system runs on a local server, and the usability test is moderated and conducted on a laptop.

This project was limited to what was achievable in the 12-week FYP II time frame. The data was prepared and the model trained. Then the web application was developed, the LLM integrated, the project tested and documented.

1.7 Significance of Research

This project aids QA engineers and student testers by using the unstructured data to create a prioritised and explained report. This will help the team decide which defects to fix first based on the predicted severity. The recommendation will provide a starting point for fixing defects. From a researcher perspective, this project provides a documented baseline with results averaged at the macro level. It uses a public dataset and compares to a majority class baseline. From a university perspective it shows how data analytics, web engineering and generative AI can be used together to produce a deliverable.

Contributions made by this study include: (a) An XGBoost severity model with documented methods for dealing with imbalance, (b) An integrated web application providing structured recommendations using LLMs, (c) An evaluation combining machine learning metrics, automated functional testing, and a usability survey.

1.8 Thesis Structure

The dissertation consists of 5 chapters. Chapter 1 provides an overview of the research problem, aims, and scope. Chapter 2 provides a literature review and identifies the research gap. Chapter 3 describes the methodology used, the data, model, system design, and evaluation. Chapter 4 demonstrates and discusses the results from the evaluation of the model, system, and usability. Chapter 5 concludes the study and provides recommendations for future work.

CHAPTER 2
LITERATURE REVIEW AND THEORY

2.1 Introduction

This chapter reviews the concepts and studies which support IntelliBug. Section 2.2 explains the theory behind bug severity, text representation, gradient boosting, imbalance handling, and usability measurement. Section 2.3 reviews related works. Section 2.4 states the research gap, and Section 2.5 summarises the chapter.

2.2 Theoretical Framework

2.2.1 Bug Reports and Severity

A bug report is a structured record of a failure observed in a software product. Severity describes the technical impact of the defect, while priority describes the urgency of the repair. In Bugzilla, severity takes one of seven values: blocker, critical, major, normal, minor, trivial or enhancement [1]. Prior studies treat the field as a classification target and report a noisy label, since reporters often keep the default value [2]. The Eclipse and Mozilla defect tracking dataset was released to support research of this kind [9].

2.2.2 Text Preprocessing and TF-IDF

Text must be converted into numbers before a classifier is able to use it. Preprocessing lowers the case, removes symbols and numbers, splits the text into tokens and removes stop words, which are familiar words with little meaning. The cleaned tokens are then weighted using term frequency–inverse document frequency (TF-IDF). A term has a high weight when it is frequent in one document and rare in the corpus [10], [11].

The scikit-learn implementation [12] with default settings computes the inverse document frequency as in Equation (1) and the weight as in Equation (2), where N is the number of documents and df(t) is the number of documents which contain term t. Each document vector is then normalised to unit length.

$$\mathrm{idf}(t) = \ln\left(\frac{1+N}{1+\mathrm{df}(t)}\right) + 1 \tag{1}$$

$$\mathrm{tf-idf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t) \tag{2}$$

2.2.3 Gradient Boosting and XGBoost

Gradient boosting builds a strong model from many weak decision trees. Each new tree is fitted to correct the errors of the trees before it [13]. XGBoost is an optimised implementation which adds a regularisation term, a sparsity-aware split search and fast histogram-based training [4]. These properties suit TF-IDF matrices, which are large and mostly zero.

The model minimises the objective in Equation (3), where l is the loss between the prediction and the true label, f_m is the m-th tree and M is the number of trees. The complexity of a tree is penalised by Equation (4), where T is the number of leaves, w is the vector of leaf weights, and γ and λ are regularisation constants.

$$L = \sum_{i=1}^{n} l(\hat{y}_i, y_i) + \sum_{m=1}^{M} \Omega(f_m) \tag{3}$$

$$\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2 \tag{4}$$

2.2.4 Class Imbalance and Evaluation Metrics

When one class dominates the data, a classifier tends to favour it. A common remedy is cost-sensitive learning, in which rare classes receive a larger weight during training [8]. The balanced scheme of scikit-learn gives class c the weight in Equation (5), where n is the number of training samples, K is the number of classes and n_c is the number of samples of class c.

$$w_c = \frac{n}{K n_c} \tag{5}$$

Accuracy is misleading on imbalanced data, so precision, recall and the F1-score are computed for each class, using Equations (6) to (8), and averaged over the classes without weighting, as in Equation (9). The macro average gives each severity class the same importance. Sokolova and Lapalme [14] show macro-averaging to treat all classes equally, while micro-averaging favours the larger classes.

$$P_c = \frac{TP_c}{TP_c + FP_c} \tag{6}$$

$$R_c = \frac{TP_c}{TP_c + FN_c} \tag{7}$$

$$F1_c = \frac{2P_c R_c}{P_c + R_c} \tag{8}$$

$$\text{Macro} - \text{F1} = \frac{1}{K}\sum_{c=1}^{K} F1_c \quad (9)$$

2.2.5 LLM-Based Solution Recommendation

A large language model will generate a response to a given prompt. If the prompt includes the bug title, description and predicted severity and asks for the answer to be written in a specific JSON format then it can be parsed by program code to provide structured output [5]. As a language model may provide misleading advice a suggestion is provided to the tester to confirm. The system will still function if the model service is not available.

2.2.6 Usability Evaluation with the System Usability Scale

The System Usability Scale (SUS) is a ten-item questionnaire with five response options [15]. The score is computed using Equation (10), in which x_i is the answer to item i. The result ranges from 0 to 100. Studies of the scale report high reliability [16], and a mean of about 68 is commonly used as the average benchmark [17]. Small groups of five users reveal most usability problems of a task-based test [18].

$$\text{SUS} = 2.5[\sum_{i\in odd}(x_i - 1) + \sum_{i\in even}(5 - x_i)] \quad (10)$$

2.3 Related Works

2.3.1 Bug Severity Prediction

Menzies and Marcus [3] applied text mining and rule learning to defect reports of NASA projects and showed severity to be predictable from the words of a report. Lamkanfi et al. [2] used a Naive Bayes classifier on the short descriptions of Eclipse, Mozilla, and GNOME reports to separate severe from non-severe bugs. A follow-up study compared four classification algorithms and reported the best results for Naive Bayes Multinomial on the data used [6]. Tian et al. [7] applied an information-retrieval based nearest neighbour method to predict fine-grained severity levels in Bugzilla repositories.

These studies establish a signal for severity in the title and the description. They also show a common pattern: the output is a label, the data are noisy, and the evaluation is offline. Most of them predict two groups, so their scores are not directly comparable with a three-class problem.

Later studies extended the features and the models. Zhang et al. [19] combined a topic-based similarity measure with k-nearest neighbours to predict severity and to recommend a fixer on five open-source projects. Ramay et al. [20] added an emotion score of the report to a deep neural network and reported an average gain of 7.90% in F-measure over earlier methods. Kukkar et al. [21] combined a convolutional neural network with a boosted random forest and reported an average accuracy of 96.34% on five projects. Tan et al. [22] enriched bug reports with question-and-answer posts from Stack Overflow and raised the average F-measure of a Naive Bayes baseline by 20.59% to 23.03% on Mozilla, Eclipse and GCC. Uddin et al. [23] surveyed the prioritisation methods and grouped them into classification, information retrieval and clustering.

Recent studies from 2024 to 2026 mostly use deep learning. Mian and Alsaeedi [24] trained CNN and BiLSTM models with GloVe embeddings on Eclipse and Mozilla reports, and reported better results for BiLSTM, with 91% accuracy on Mozilla and 89% on Eclipse. Ali et al. [25] fine-tuned BERT with the sentiment of the reporter on a public dataset of mobile application bug reports. Arshad et al. [26] fine-tuned GPT-2 with sentiment features on 16,512 Eclipse and Mozilla reports in two classes and reported an accuracy of 76.8% under leave-one-project-out validation. Sarawan et al. [27] combined TF-IDF with Word2Vec, FastText or BERT embeddings for five severity levels of Mozilla reports, and reported accuracy near 66% for logistic regression. Zhu et al. [28] applied DistilBERT and TinyBERT with domain alignment to the titles of 49,354 Mozilla reports in five levels and reported a macro F1-score of 0.56 for DistilBERT with alignment.

Two points follow from these studies. First, the scores depend on the task, and accuracy and macro F1-score are not directly comparable. The five-level, cross-project study of Zhu et al. reports a macro F1-score of 0.56 at best, which gives context to the 0.365 of IntelliBug on a three-class task with a strong imbalance. Second, all five studies stop at the severity label. None links the prediction to repair advice or wraps the model in a tool with tracking and evaluation.

2.3.2 Bug Triage and Defect Management Support

Anvik et al. [29] built a classifier to suggest the best developer to fix a bug based on training data from Eclipse and Firefox. This shows how machine learning can help

with triage, however it does not deal with severity or suggestions for fixing the bug. Commercial issue trackers store and filter issues, and they leave the severity to the reporter.

2.3.3 Large Language Models in Software Engineering

Large language models are now applied across the software life cycle. Hou et al. [30] reviewed 395 studies published from January 2017 to January 2024 and mapped the models, the data and the software engineering tasks in which they are used. The review reports the understanding of the effects and the limits of these models to be at an early stage. Xia et al. [31] evaluated nine pre-trained language models on five repair datasets in three programming languages, and the models outperformed the earlier automated program repair techniques on all datasets. These results support the use of a language model to draft repair advice. They also show a limit for IntelliBug. The repair studies give the model the faulty source code, while IntelliBug gives the model the text of the report only. The advice is therefore general, and a tester needs to confirm it.

2.3.4 Summary of Related Studies

Table 2.1 compares the reviewed studies with IntelliBug.

Table 2.1: Comparison of related studies and the proposed system

Study	Data	Technique	Output	Limitation
Menzies and Marcus (2008)	NASA defect reports	Text mining, rule learner	Severity level	Label only; no tool
Lamkanfi et al. (2010)	Eclipse, Mozilla, GNOME	Naive Bayes	Severe or non-severe	Two classes; offline
Lamkanfi et al. (2011)	Open-source Bugzilla data	Four classifiers compared	Severe or non-severe	Two classes; offline
Tian et al. (2012)	Bugzilla repositories	Nearest neighbour retrieval	Fine-grained severity	Label only; no tool
Anvik et al. (2006)	Eclipse, Firefox	Supervised classifier	Developer assignment	No severity or fix advice
Zhang et al. (2016) [19]	GCC, OpenOffice, Eclipse, NetBeans, Mozilla	Topic model with k-nearest neighbours	Severity level and fixer	No fix advice; no tool
Ramay et al. (2019) [20]	Bug report history, cross-product	Deep neural network with emotion score	Severity level	Label only; no tool
Kukkar et al. (2019) [21]	Mozilla, Eclipse, JBoss, OpenFOAM, Firefox	CNN with boosted random forest	Severity level	Label only; no tool
Tan et al. (2020) [22]	Mozilla, Eclipse, GCC with Stack Overflow posts	Logistic regression on enriched reports	Severity level	Label only; no tool
Mian and Alsaeedi (2024) [24]	Eclipse, Mozilla	CNN, BiLSTM, GloVe	Severity level	Label only; no tool
Ali et al. (2024) [25]	Mobile app bug reports	Fine-tuned BERT, sentiment	Severity level	Label only; no tool
Arshad et al. (2024) [26]	Eclipse, Mozilla	Fine-tuned GPT-2, sentiment	Severe or non-severe	Two classes; no tool

Sarawan et al. (2025) [27]	Mozilla	TF-IDF with embeddings; LR, SVM, RF	Five severity levels	Label only; no tool
Zhu et al. (2026) [28]	Mozilla, 49,354 reports	DistilBERT, TinyBERT, domain alignment	Five severity levels	Label only; no tool
IntelliBug (this study)	Eclipse, 478,653 reports	TF-IDF, XGBoost, Claude LLM	Three severity classes, fix advice and dashboard	Title-only training; low minority precision

Three observations follow from Table 2.1. First, the reported scores are not comparable at face value. Studies which report accuracy reach high values, such as the 96.34% of Kukkar et al. [21], while the one study which reports a macro F1-score on five levels reaches 0.56 [28]. Accuracy rewards the majority class, so a study which reports accuracy alone hides the errors on the rare and costly classes [8], [14]. Second, the models grew from Naive Bayes to transformers, yet the input stayed the same: the short text of the report, with a label assigned by the reporter. Bettenburg et al. [32] found a mismatch between the information developers need and the information reporters provide, so the quality of the report limits every model. Third, every study ends at the prediction. None measures whether a tester is able to act on the output, and none evaluates a working tool with users.

This study responds to each observation. It reports macro-averaged scores with a majority-class baseline, it keeps the natural class distribution in the test set, and it evaluates the complete tool with functional tests, a rating study and a usability test.

2.4 Research Gap

This review found four limitations that IntelliBug helps to overcome:

1. Severity prediction is not linked to recommendations for fixing the issue. None of the studies in the review mentioned how the severity is used to make recommendations for fixing the problem.
2. The models developed are rarely implemented in a tool that allows for monitoring, filtering, and analysis.

3. The accuracy of the models is often reported without regard to the class imbalance. Therefore, the results should be presented using macro averaging and a baseline.
4. There are few studies that use functional testing or user centred testing to test the tool.

These limitations are important to overcome because the severity label will only have value if it impacts the next steps of the team.

2.5 Chapter Summary

This chapter explained bug severity, TF-IDF, XGBoost, imbalance handling, LLM recommendation and SUS. The related studies show severity to be predictable from text, and the repair studies show language models to produce useful fixes. An integrated and evaluated tool is missing. The next chapter describes how IntelliBug was built to fill this gap.

CHAPTER 3
METHODOLOGY AND PROJECT WORK

3.1 Introduction

The project follows a development and experimental research design. The machine learning part follows the CRISP-DM process [33], and the web application part follows an iterative Agile approach [34], in which small modules were built, assessed, and refined. Figure 3.1 shows the ten phases of the project.

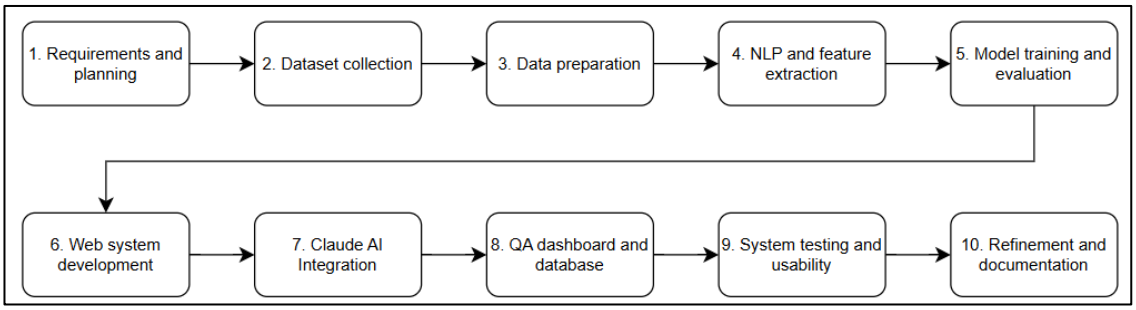


Figure 3.1: Project methodology and phases

Table 3.1 lists the deliverable of each phase.

Table 3.1: Project phases and deliverables

Phase	Activity	Deliverable
1	Requirements and planning	Requirements list; development environment
2	Dataset collection	Eclipse Bugzilla export (xlsx)
3	Data preparation	Cleaned dataset with severity labels
4	NLP and feature extraction	Preprocessed text; TF-IDF features
5	Model training and evaluation	Trained XGBoost model; metrics
6	Web system development	Flask application with login
7	Claude AI integration	Structured recommendation module
8	QA dashboard and database	Dashboard, reports, analytics; MySQL
9	System testing and usability	Selenium suite; SUS results
10	Refinement and documentation	Final system; dissertation

3.2 Dataset

3.2.1 Data Source

The dataset is a snapshot of the Eclipse Bugzilla repository on 22 July 2022. It contains approximately 559,000 entries, and includes bug id, brief description (title), severity, product, and component. The repository is part of a family of datasets [9].

3.2.2 Cleaning and Severity Mapping

Four cleaning steps were applied. Records with the severity enhancement were removed because a feature request is not a defect. Records with invalid severity values, missing text or duplicate identifiers were also removed. The six remaining severity values were then grouped into three classes, as shown in Table 3.2.

Table 3.2: Severity mapping used in this project

Bugzilla severity	IntelliBug class
blocker, critical	Critical
major, normal	Medium
minor, trivial	Low

After cleaning, 478,883 records remained. After text preprocessing, 230 records were removed because no word was left, which gave 478,653 records for modelling.

3.3 Text Preprocessing and Feature Extraction

The title of each report was lowercased. Text in brackets, for example internal reference codes, was removed, and every character except letters and spaces was replaced by a space. The text was tokenised with NLTK, and English stop words were removed. The remaining tokens were joined again and converted to a TF-IDF matrix with a vocabulary limited to 10,000 terms (Equations 1 and 2). The vocabulary was raised from 5,000 to 10,000 terms during tuning to keep more severity-related words. The pipeline is shown in Figure 3.2.

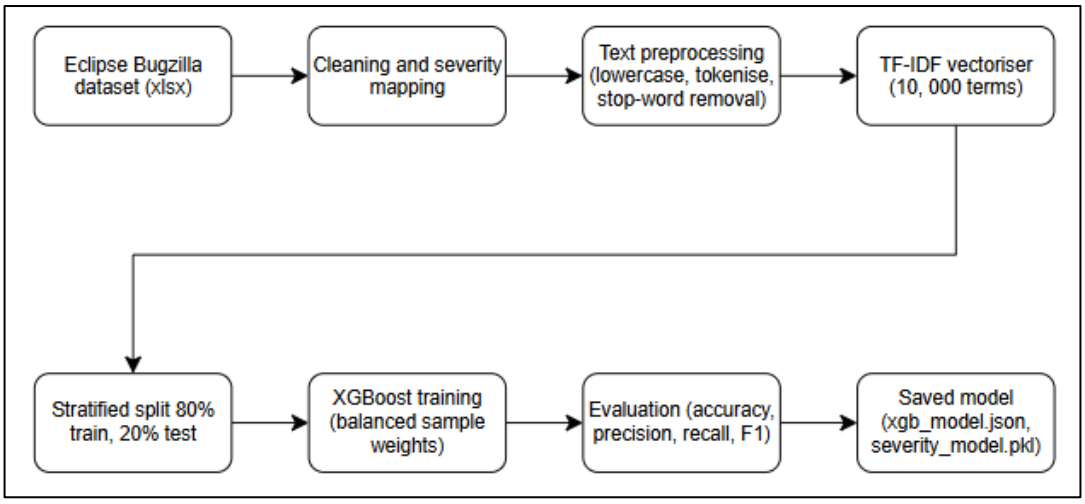


Figure 3.2: Machine learning pipeline

The model was trained on the title only. At prediction time, the system joins the title and the description entered by the user and applies the same cleaning steps. This difference between training and use is discussed as a limitation in Chapter 4.

3.4 Model Development

3.4.1 Data Split

The records were divided into 80% for training and 20% for testing with stratified sampling and a fixed random seed of 42. The training set holds 382,922 records and the test set holds 95,731 records. Stratification keeps the class proportions equal in both sets.

3.4.2 Class Imbalance

Medium reports contain around 89% of the data. The models were trained using balanced weights (Equation 5) meaning that errors on Critical and Low reports were weighted higher than errors on medium reports [8]. Weights were used instead of re-labelling or sampling as we retain all the original records.

3.4.3 XGBoost Configuration

The classifier was configured with the hyperparameters in Table 3.3. The multi-class softmax objective was used with histogram-based tree construction for speed on the sparse matrix.

Table 3.3: XGBoost hyperparameters

Hyperparameter	Value
n_estimators	300
max_depth	7
learning_rate	0.08
subsample	0.8
colsample_bytree	0.8
tree_method	hist
objective	multi:softmax (3 classes)
eval_metric	mlogloss
random_state	42

Three tuning experiments were run: a larger vocabulary, the addition of one-hot encoded product and component fields, and a search of the hyperparameters. The trained model was saved as a JSON file for the booster and a separate pickle file for the label encoder and the vectoriser. This split-file format avoids pickle corruption of the booster on Windows.

3.5 System Design

3.5.1 Architecture

IntelliBug is a Flask [35] application with six pages. Figure 3.3 shows the main parts. The browser talks to Flask, which authenticates the user through Google, classifies the report with the XGBoost model, requests a recommendation from the Claude API, and stores the result in MySQL. Screenshots are stored as files with random names.

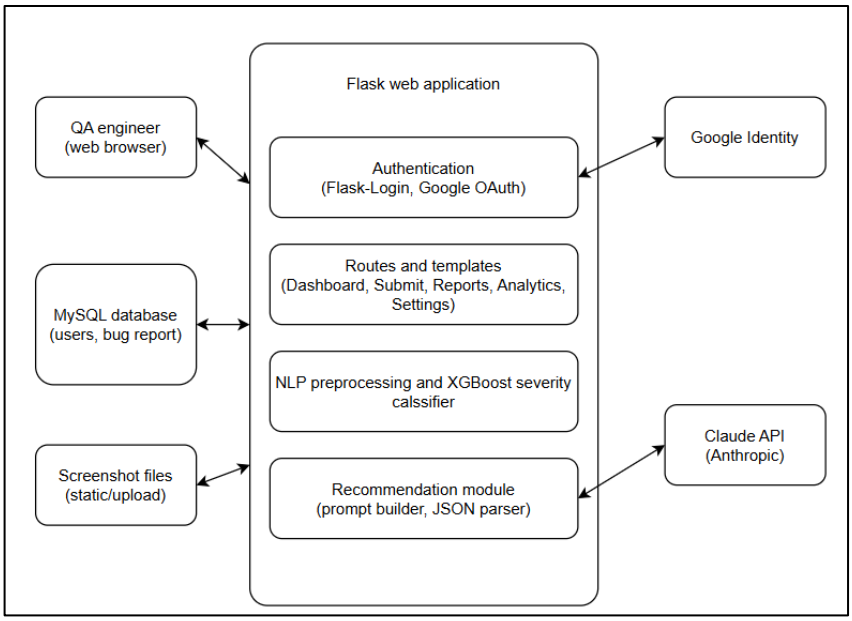


Figure 3.3: System architecture of IntelliBug

3.5.2 Use Case Diagram

Figure 3.4 shows the use case diagram. The QA tester is the main actor and reaches nine use cases, from sign-in to log-out. Submitting a bug report includes two internal use cases: the severity prediction by the XGBoost model and the generation of the recommendation. Google Identity supports the sign-in, and the Claude API supports the recommendation.

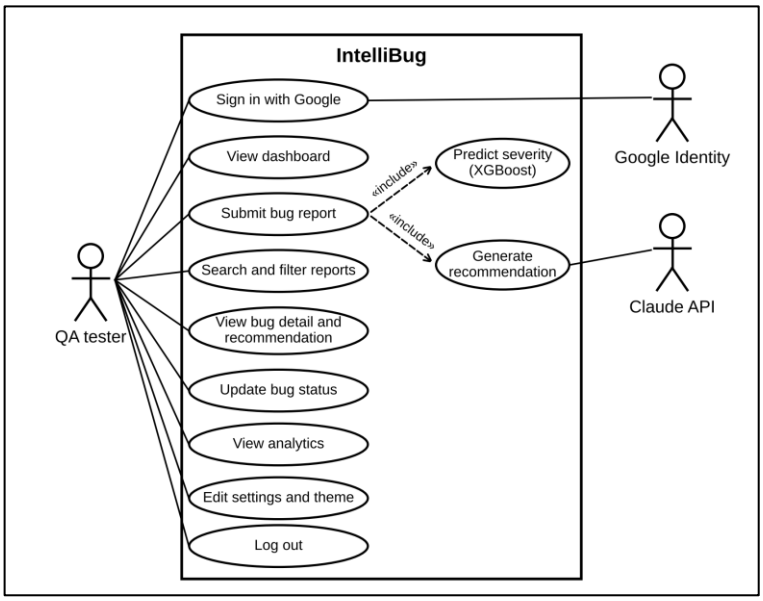


Figure 3.4: Use case diagram of IntelliBug

3.5.3 Sequence Diagram

Figure 3.5 shows the order of the messages when a tester submits a bug report. The server checks the fields and saves the screenshot, converts the cleaned text to TF-IDF features and asks the model for the severity class. It then sends the title, the description and the severity to the Claude API and receives the recommendation as JSON. When the call fails, a fallback message is stored. The report is saved in MySQL, and the result page shows the severity and the three-step recommendation.

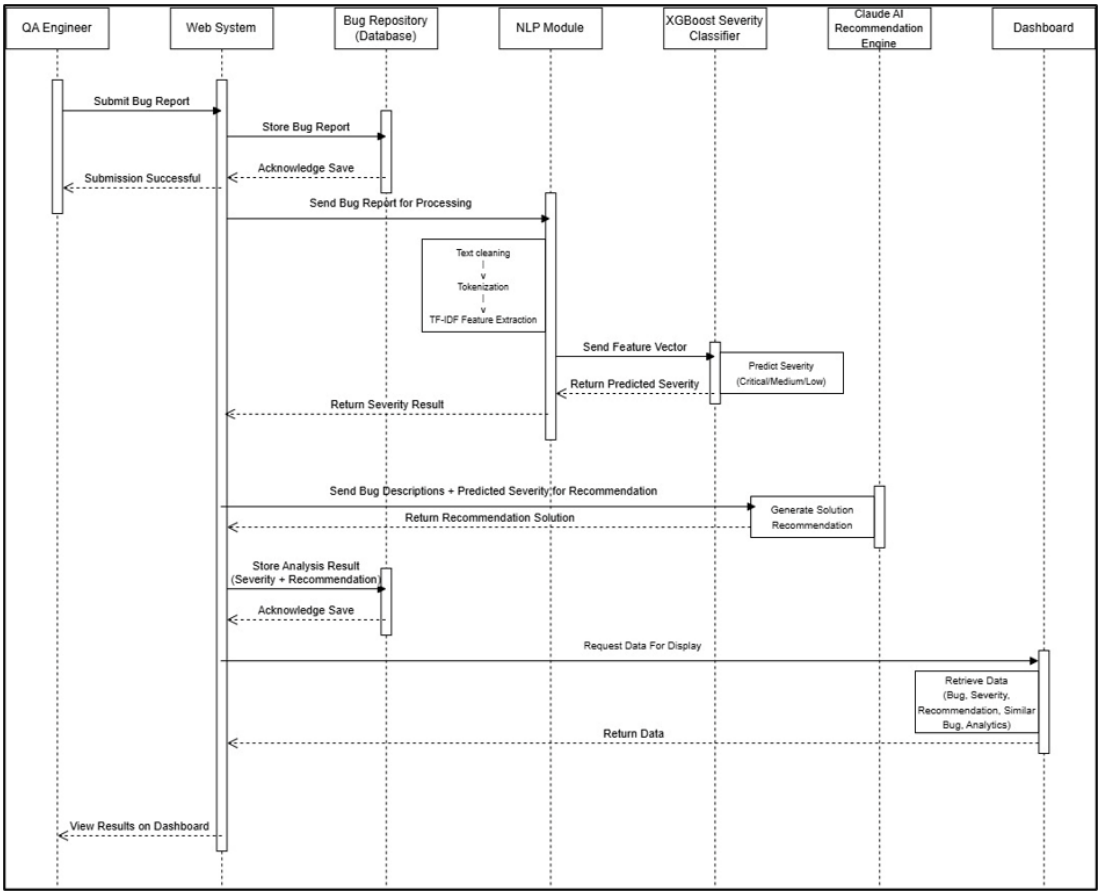


Figure 3.5: Sequence diagram of the bug submission

3.5.4 Database Design

There are two tables in the database: users and bug_reports. These are in a one-to-many relationship. Both tables can be accessed using SQLAlchemy. This means the same code can be used when using MySQL with XAMPP and SQLite for local testing.

The main fields can be seen in tables 3.4 and 3.5. Figure 3.6 shows the entity-relationship diagram.

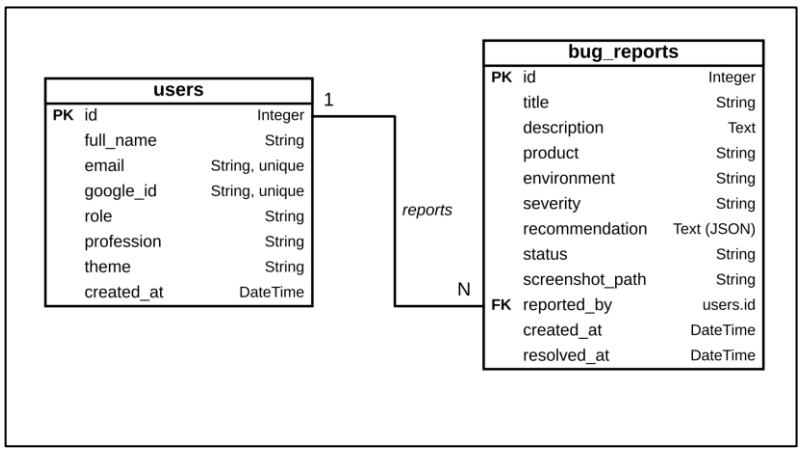


Figure 3.6: Entity-relationship diagram of the database

Table 3.4: Fields of the users table

Field	Type	Description
id	Integer, primary key	User identifier
full_name, email	String	Name and unique email from Google
google_id	String, unique	Google account identifier
role, profession, theme	String	Profile and display settings
created_at	DateTime	Account creation time

Table 3.5: Fields of the bug_reports table

Field	Type	Description
id	Integer, primary key	Report identifier
title, description	String, Text	Text entered by the user
product, environment	String	Affected module and test environment
severity	String	Predicted class: Critical, Medium, or Low
recommendation	Text (JSON)	Root cause, suggested fix, best practice
status	String	Open, In Progress or Resolved
screenshot_path	String	Relative path of the uploaded image
reported_by	Foreign key to users.id	Owner of the report

Field	Type	Description
created_at, resolved_at	DateTime	Submission and resolution times

3.5.5 Recommendation Module

After the severity is predicted, the title, the description and the predicted severity are placed in a prompt and sent to the Claude model claude-sonnet-4-6 with a limit of 300 output tokens. The prompt asks for a JSON object with three short sentences: root_cause, suggested_fix, and best_practice. The reply is cleaned of any code fences, parsed, stored in the database, and shown as a three-step diagram. When the call fails or the reply is not valid JSON, a fallback message is stored, and the severity result is still shown [5].

3.5.6 Security and Privacy

Controls have been put in place. All pages (except login) require a logged in session using Flask-Login. The login process uses Google OAuth. All queries to the bug reports database are filtered using the logged in users ID. This means that only the reports that the user has submitted will be shown. If a user tries to view the address of other users report they will be redirected to their own list. All uploaded images are checked to make sure they are an image extension and saved with a random name. All pages have cache headers set to no-store. This means that if a user has viewed a page before logging out they will not be able to view it again by pressing the back button. The login route for automated tests is only available if the environment variable TESTING is set to 1.

3.6 System Implementation

Table 3.6 describes the pages of the system.

Table 3.6: Pages and functions of IntelliBug

Page	Function
Login	Google sign-in
Dashboard	Open bug counts by severity; five recent reports; recent screenshots; latest Critical recommendation
Submit Bug	Form with title, product, environment, description, and screenshot, all required; shows severity and the three-step recommendation

Page	Function
Bug Reports	List with search by title and filters for severity and status
Bug Detail	Full report, screenshot, recommendation, highest-weight TF-IDF terms, status update
Analytics	Weekly trend of the last eight weeks, open bugs by product, mean resolution time, model metrics
Settings	Name, role, profile picture and light or dark theme

3.7 Testing and Evaluation Plan

3.7.1 Machine Learning Evaluation

The model is evaluated on the held-out test set using accuracy and the macro-averaged precision, recall, and F1-score (Equations 6 to 9), with a confusion matrix. A majority-class baseline is added to show the effect of the imbalance, and the tuning experiments are compared on macro F1.

3.7.2 Functional Testing

Functional testing was carried out using Selenium WebDriver [36]. This was done using pytest and the Page Object pattern. The tests open Chrome, log in using a test only route and simulate user interaction with the pages. All waits were explicit and an HTML report was generated that included a screenshot for each test. There were twelve tests, covering logging in, submitting a bug, mandatory fields, changing status, back button, logging out and user privacy.

3.7.3 Recommendation Evaluation

The quality of the recommendations was rated by five raters, who were friends of the author. The author submitted ten bug reports of different severities to IntelliBug: three Critical, one Medium and six Low. Each report received one three-step recommendation made of a root cause, a suggested fix, and a best practice. Each recommendation was scored by each rater using a scale of 1-5. Each recommendation was evaluated based on three criteria: Relevance (does the recommendation address the issue?), Correctness (are the underlying cause and solution technically correct?) and Actionable (can a developer take action to resolve the issue without any additional information?). A score of 1 means a poor recommendation whilst a score of 5 means

an excellent recommendation. Table 4.8 reports the mean and the standard deviation of each criterion over all 50 ratings.

3.7.4 Usability Evaluation

This is a moderated usability study. Each user will sit next to the researcher on a laptop with a locally running system and complete the 4 tasks in Table 3.7. They will then complete the 10 SUS questions and two optional open questions on an anonymous Google Form. The metrics that will be collected are task completion rate and average SUS score.

Table 3.7: Usability test tasks

Task	Instructions to the participant	Success criterion
1. Log in	Sign in to IntelliBug with Google.	Dashboard is reached
2. Submit a bug	Report that the Login button does nothing on Safari. Fill in every field and attach a screenshot.	Severity and the three-step recommendation appear
3. Find and read	Find the bug in Bug Reports, open it, and read the suggested fix.	Suggested fix is read out
4. Update status	Mark the bug as Resolved.	Status badge shows Resolved

3.8 Tools and Technologies

Table 3.8 lists the tools used to develop, test and document IntelliBug. The paragraphs after the table explain the role of each group of tools.

Table 3.8: Tools and technologies

Area	Tool
Programming language	Python 3.13
Web framework	Flask, Flask-Login, Flask-SQLAlchemy, Authlib
Machine learning	XGBoost, scikit-learn, NLTK, pandas, joblib
Generative AI	Claude API (claude-sonnet-4-6)
Database	MySQL (XAMPP, phpMyAdmin), SQLite for tests
Authentication service	Google OAuth 2.0, Google Cloud Console
Front end	HTML, CSS, JavaScript, Chart.js
Testing	Selenium WebDriver, pytest, pytest-html

Area	Tool
Survey	Google Forms, Microsoft Excel
Version control	Git and GitHub
Environment	Windows, PowerShell, Visual Studio Code, virtual environment
Documentation	Microsoft Word

Development environment. The system was developed on a Windows laptop with Visual Studio Code as the code editor and PowerShell as the terminal. A Python virtual environment kept the project libraries separate from the system installation. Git recorded each change, and the code was stored in a GitHub repository (https://github.com/leehana312/IntelliBug-FYP).

Programming language and web framework. Python 3.13 was the single language for the model and the server. Flask handled the routes and the HTML templates. Flask-Login managed the sessions, Authlib handled the Google OAuth 2.0 sign-in, and Flask-SQLAlchemy mapped the Python classes to the database tables. The OAuth client was registered in the Google Cloud Console, and the keys were kept in a local .env file loaded with python-dotenv. The pages were written in HTML, CSS and JavaScript, and Chart.js drew the charts on the Analytics page.

Machine learning. pandas loaded and cleaned the Eclipse export. NLTK tokenised the titles and supplied the English stop-word list. scikit-learn provided the TF-IDF vectoriser, the stratified split, the balanced sample weights and the evaluation metrics. XGBoost trained the classifier, and joblib saved the vectoriser and the label encoder.

Generative AI. The Claude API of Anthropic, with the model claude-sonnet-4-6, generated the recommendation for each report. The Python library of the API sent the prompt and returned the JSON reply.

Database. MySQL stored the users and the bug reports. XAMPP ran the MySQL server on the laptop, and phpMyAdmin was used to inspect the tables. SQLite replaced MySQL during the automated tests with no change to the code.

Testing and evaluation. Selenium WebDriver controlled the Chrome browser, pytest ran the twelve test cases, and pytest-html produced the report in Appendix A. Google

Forms collected the SUS answers, and Microsoft Excel computed the SUS scores and the rating statistics. Microsoft Word was used to write this dissertation.

3.9 Project Timeline and Key Milestones

The work was planned over the 12 weeks of FYP II. The plan has eleven activities, listed in Table C.1 of Appendix C, and Figure C.1 shows them as a Gantt chart. Each activity ends in a deliverable which the next activity needs, so progress was checked each week against the plan. Table 3.9 lists the key milestones.

Table 3.9: Key milestones of FYP II

No.	Key milestone	Week
1	Requirements and project plan confirmed	1
2	Dataset collected, cleaned and labelled	2
3	Text preprocessing and TF-IDF features complete	3
4	XGBoost model trained	5
5	Model tested and evaluated	6
6	Web system working with Google sign-in	8
7	Claude recommendation integrated	9
8	QA dashboard and analytics complete	10
9	System integration and functional testing complete	11
10	Refinement, dissertation and presentation complete	12

The model work filled the first six weeks, and the system work, the integration and the testing filled the last six. All milestones were reached within the 12 weeks, which shows the methodology to be achievable in the time frame of the project.

3.10 Chapter Summary

This chapter described the development and experimental design, the Eclipse Bugzilla dataset, the preprocessing, and TF-IDF features, the XGBoost model with balanced weights, the architecture and database of IntelliBug, the recommendation module, and the plan for model, functional and usability evaluation. It also described the use case, sequence and entity-relationship diagrams, the tools and the 12-week plan. The next chapter reports the results.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the results in the order of the objectives: data and model results (Sections 4.2 and 4.3), system implementation (Section 4.4), functional testing (Section 4.5), recommendation quality (Section 4.6), and usability (Section 4.7). Section 4.8 discusses the findings against the objectives, the literature and the theory.

4.2 Data Characteristics

Table 4.1 shows the number of records after each preparation step, and Figure 4.1 shows the class distribution of the test set.

Table 4.1: Number of records after each preparation step

Step	Records
Raw Eclipse Bugzilla export	approx. 559,000
After cleaning and severity mapping	478,883
After text preprocessing (modelling set)	478,653
Training set (80%)	382,922
Test set (20%)	95,731

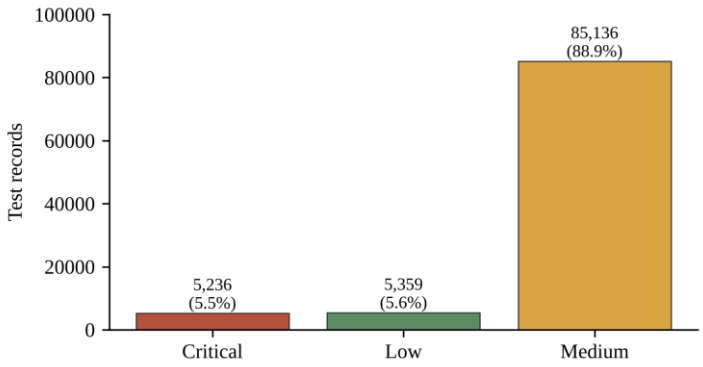


Figure 4.1: Class distribution of the test set

Medium reports form 88.9% of the test set, Critical reports 5.5% and Low reports 5.6%. The imbalance justifies both the balanced sample weights and the use of macro-averaged metrics.

4.3 Model Performance

4.3.1 Overall Results

Table 4.2 gives the metrics of the deployed text-only model on the test set. These values are also shown on the Analytics page of the system.

Table 4.2: Overall performance of the deployed model

Metric	Value
Accuracy	55.5%
Macro precision	38.9%
Macro recall	52.6%
Macro F1-score	36.5%

4.3.2 Confusion Matrix and Per-Class Results

Figure 4.2 shows the confusion matrix of the deployed model, and Table 4.3 gives the per-class scores computed from it.

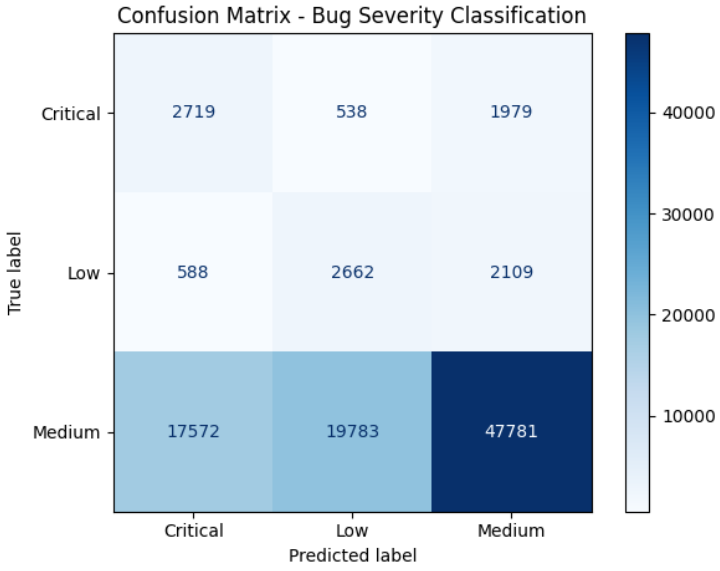


Figure 4.2: Confusion matrix on the test set

Table 4.3: Per-class results computed from the confusion matrix

Class	Precision	Recall	F1-score	Support
Critical	13.0%	51.9%	20.8%	5,236
Low	11.6%	49.7%	18.8%	5,359
Medium	92.1%	56.1%	69.8%	85,136

Class	Precision	Recall	F1-score	Support
Macro average	38.9%	52.6%	36.5%	95,731

The macro average reproduces Table 4.2 exactly, which confirms Figure 4.2 as the matrix of the deployed model.

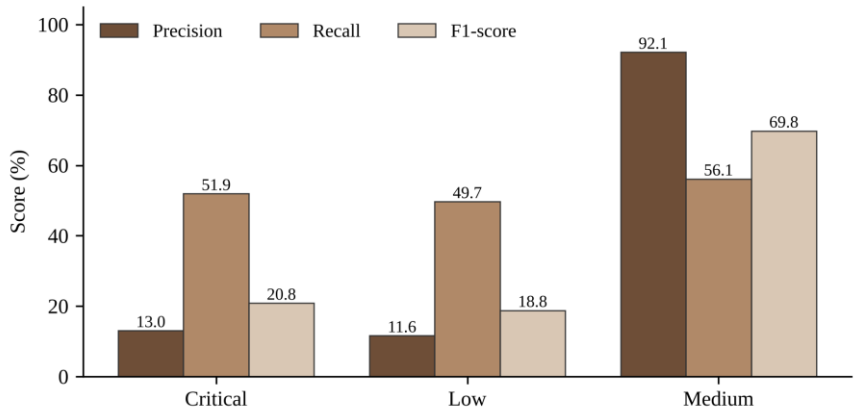


Figure 4.3: Precision, recall, and F1-score per class

About half of the Critical reports (51.9%) and of the Low reports (49.7%) were found, which means the balanced weights worked as intended. Precision is low for both classes, because many medium reports were predicted as Critical or Low: 17,572 Medium reports were labelled Critical and 19,783 were labelled Low. The Medium class has high precision (92.1%) and a recall of 56.1%.

4.3.3 Comparison with a Majority-Class Baseline

A model which always predicts Medium is the simplest baseline. Table 4.4 compares it with the XGBoost model.

Table 4.4: Comparison with a majority-class baseline

Model	Accuracy	Macro F1	Recall of Critical	Recall of Low
Always Medium	88.9%	31.4%	0%	0%
XGBoost (deployed)	55.5%	36.5%	51.9%	49.7%

Although the baseline has the highest accuracy, it does not detect any Critical or Low reports meaning it has a lower macro F1. The XGBoost model sacrifices accuracy for detection of the minority classes. This demonstrates that accuracy is not a suitable metric for this problem.

4.3.4 Tuning Experiments

Table 4.5 lists the tuning experiments of Section 3.4.3.

Table 4.5: Tuning experiments

Experiment	Change	Macro F1
1. Baseline	5,000 TF-IDF terms	35.7%
2. Larger vocabulary	10,000 TF-IDF terms	35.5%
3. Extra features	Experiment 2 plus product and component	35.7%
4. Hyperparameters	300 trees, depth 7, learning rate 0.08	36.4%

A larger vocabulary and the extra product and component fields did not raise the macro F1, which stayed between 35.5% and 35.7%. The tuned hyperparameters gave the highest value, 36.4%. The deployed model therefore uses the title text only, with the tuned hyperparameters. Its macro F1 of 36.5% in Table 4.2 is 0.1 points above this run.

4.4 System Implementation Results

Figures 4.4 to 4.9 show the main pages of the implemented system. The images were captured with demonstration data.



Figure 4.4: Login page with Google authenticator

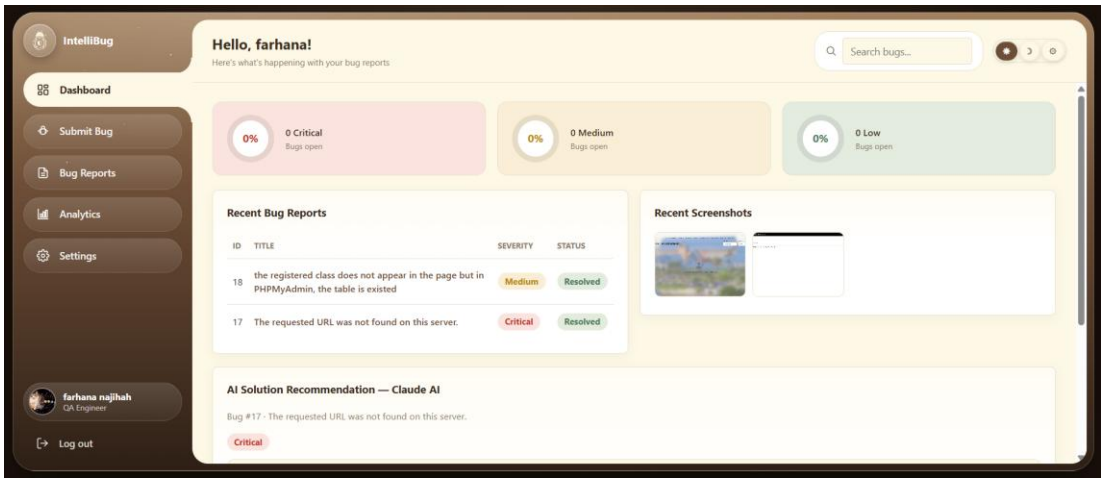


Figure 4.5: Dashboard page with recent bug reports and screenshots

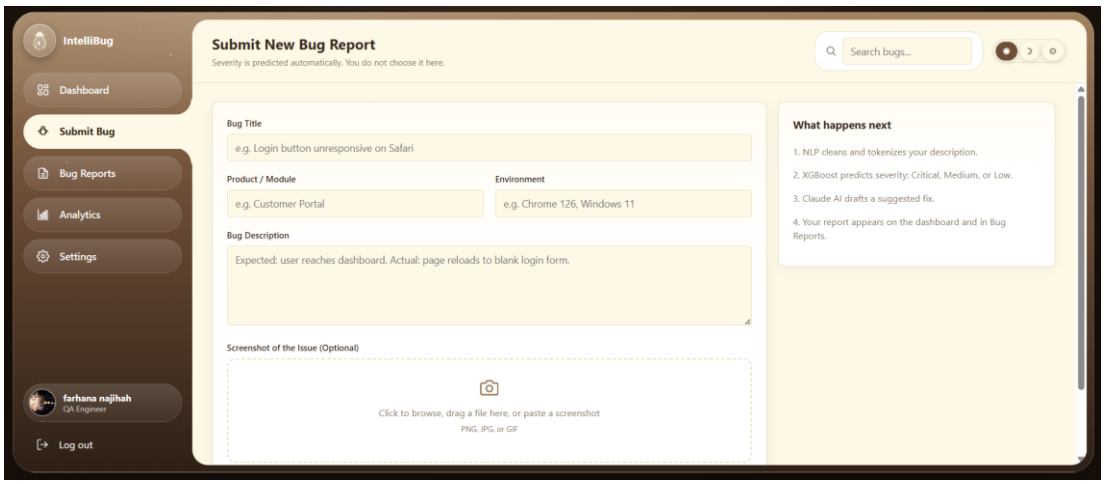


Figure 4.6: Submit Bug Reports page with some required fields to fill

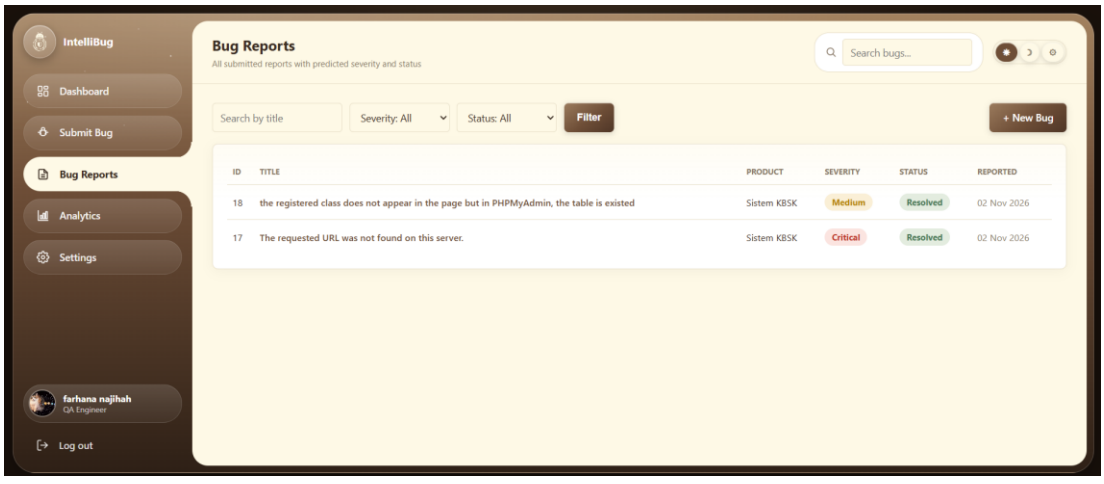


Figure 4.7: Bug Reports page with search and filters

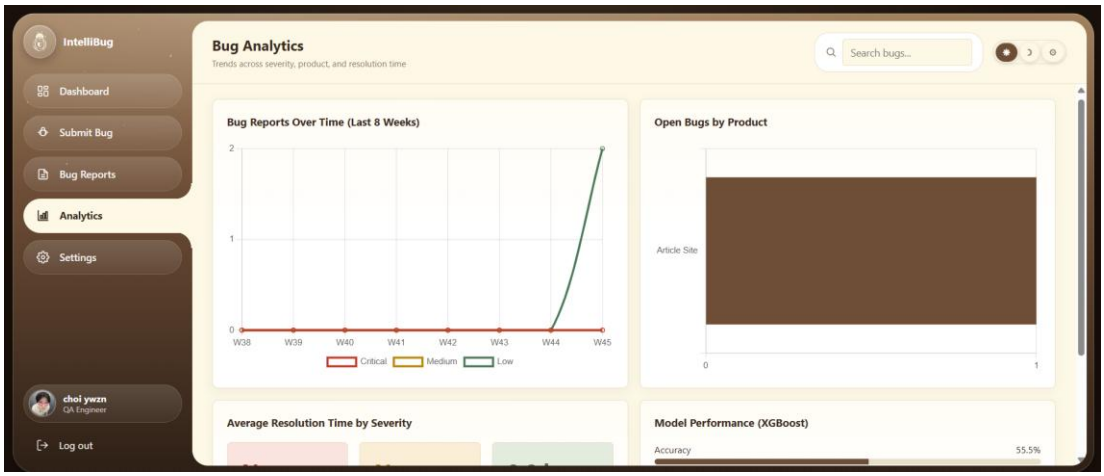


Figure 4.8: Bug Analytics page with the bug trend, open bugs by product and model performance

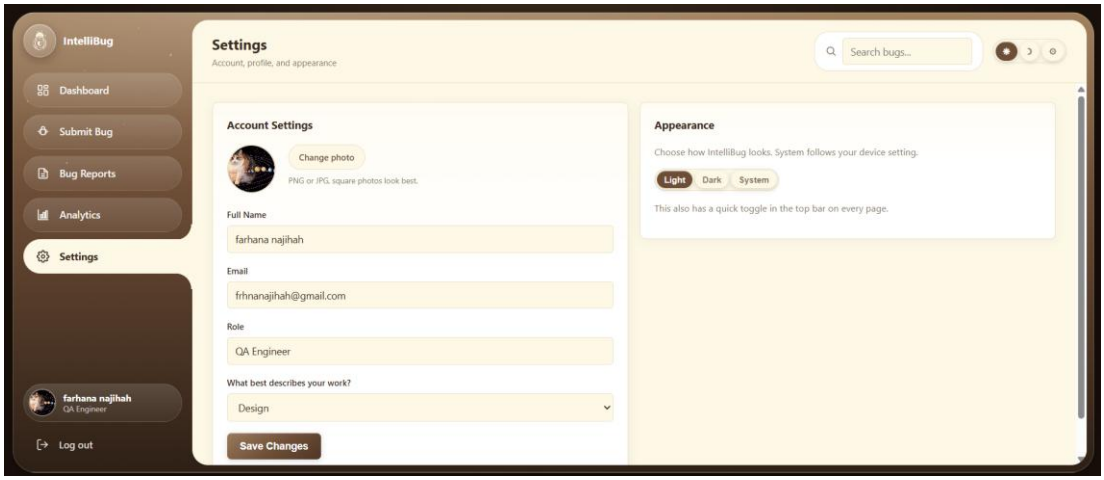


Figure 4.9: Settings page with account details and appearance options

4.5 Functional Testing Results

Twelve automated test cases were executed with Selenium and pytest. Table 4.6 lists the cases and their results.

Table 4.6: Selenium test cases and results

ID	Test case	Expected result	Result
T1	Login reaches dashboard	Dashboard page opens	Pass
T2	Submit a valid bug	Severity badge and recommendation steps appear	Pass
T3	Submit an empty form	Submission blocked; no result shown	Pass
T4	Title left empty	Browser blocks submission	Pass
T5	Product left empty	Browser blocks submission	Pass

ID	Test case	Expected result	Result
T6	Environment left empty	Browser blocks submission	Pass
T7	Description left empty	Browser blocks submission	Pass
T8	Screenshot missing	Error message asks for a screenshot	Pass
T9	Mark a bug as Resolved	Status changes to Resolved	Pass
T10	Back button after a status change	List shows the new status	Pass
T11	Logout	Login page opens	Pass
T12	Another user opens the bug	Bug is hidden from the list and its address is redirected	Pass

All 12 test cases passed in the final run, which took 1 minute 16 seconds in total, with 4 to 10 seconds for each test. The run used Python 3.13.15 and pytest 9.1.1 on Windows 11. Figure A.1 in Appendix A shows the HTML report of this run.

The testing also found defects, which were repaired and retested. Table 4.7 lists them.

Table 4.7: Defects found during testing

No.	Defect	Repair
D1	The submit form accepted empty fields and a missing screenshot.	Required attributes in the form, a screenshot check in JavaScript and server-side checks in the route.
D2	After a status change, the Back button showed the old status.	No-store cache headers on all pages except static files.
D3	A second user saw the reports of the first user.	Every query filtered by the signed-in user; direct addresses redirected.

4.6 Recommendation Quality

Five raters scored the recommendations of ten bug reports, which gives 50 ratings for each criterion. Table 4.8 gives the mean and the standard deviation of each criterion. Correctness had the highest mean at 4.34, followed by relevance at 4.20 and

actionability at 4.14. No rater gave a score below 3, and 81% of the 150 scores were 4 or 5. Figure 4.10 shows the means with the standard deviations.

Table 4.8: Mean rating of the recommendations (1 to 5)

Criterion	Mean	Standard deviation
Relevance	4.20	0.78
Correctness	4.34	0.66
Actionability	4.14	0.78

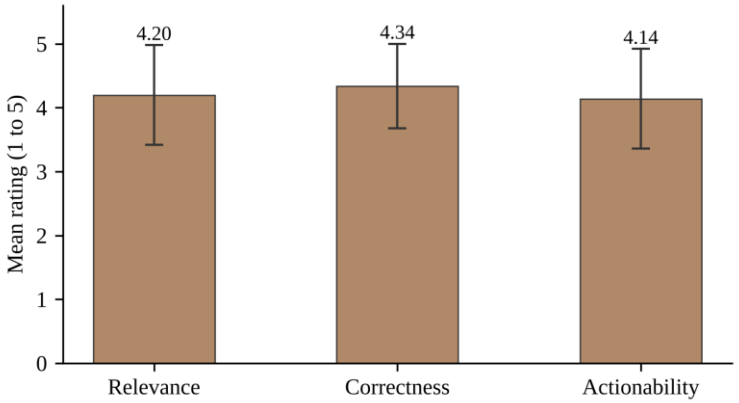


Figure 4.10: Mean rating of the recommendations with standard deviation

The scores for each report varied between 3.87 and 4.53. Report 1 and 3 had the same title but the lowest score of 3.87 and highest score of 4.53. This shows that the quality of advice given is variable. Report 5 (The requested URL was not found on this server) had the highest score for actionability at 5.0 and lowest score for correctness at 3.6. For severity the average score for the three Critical reports was 4.27, the medium report was 4.27, and the six Low reports was 4.20.

4.7 Usability Evaluation Results

Five participants completed the four tasks and the SUS questionnaire. Table 4.9 gives the task completion rates, and Table 4.10 gives the SUS scores. Figure 4.11 compares each SUS score with the benchmark.

Table 4.9: Task completion rate

Task	Completed alone	Rate
1. Log in	5 of 5	100%

Task	Completed alone	Rate
2. Submit a bug	5 of 5	100%
3. Find and read	5 of 5	100%
4. Update status	5 of 5	100%
All tasks	20 of 20	100%

Table 4.10: SUS scores of the participants

Participant	SUS score
P1	72.5
P2	70.0
P3	82.5
P4	60.0
P5	60.0
Mean	69.0

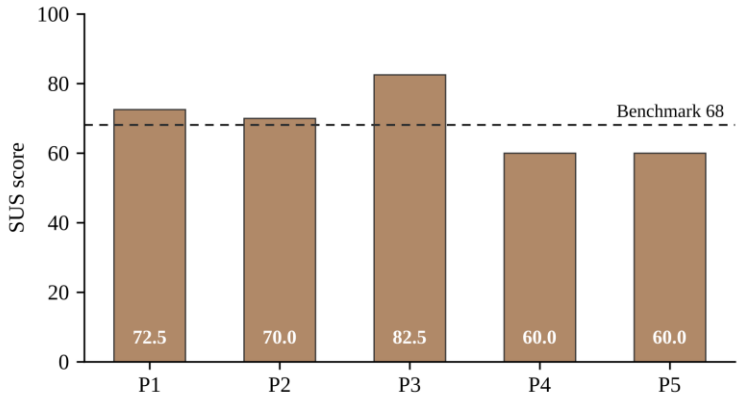


Figure 4.11: SUS score of each participant against the benchmark of 68

The mean SUS score was 69.0, with a standard deviation of 9.5, and scores ranged between 60.0 and 82.5. This is just one point above the benchmark [17]. Three out of the five participants (P1, P2, P3) had scores equal to or greater than the benchmark. A score of 69 on the adjective scale from Bangor et al. [16] can be described as OK-Good, being towards the top of this range. It should be noted that there is a small sample size. The 95% confidence interval for the mean score was 57.3 to 80.7 and so would include the benchmark score of 68. Therefore, the data do indicate that the usability is around the benchmark but cannot be confirmed as above.

Table 4.11 shows the contribution of each item. The maximum score was 4. The highest scores were for ease of use (3) and easy to learn (7), with scores of 3.4.They rated the learning effort (item 10, 1.4 points), the integration of functions (item 5, 1.8 points) and the wish to use the system frequently (item 1, 2.6 points) lowest. On item 10, two participants answered 5 and one answered 4, so three of the five felt the need to learn many things before starting. The tasks were all completed, so the lower items reflect the learning effort and the integration of the functions and not a failure of the functions.

Table 4.11: Mean item contribution to the SUS score (maximum 4)

Item	Statement (short)	Mean
1	Would use frequently	2.6
2	Not unnecessarily complex	3.0
3	Easy to use	3.4
4	No technical support needed	2.8
5	Functions well integrated	1.8
6	Little inconsistency	2.8
7	Quick to learn	3.4
8	Not cumbersome	3.2
9	Felt confident	3.2
10	Little to learn before starting	1.4

Each participant had to complete two open questions. All five participants answered both questions. When asked what they liked most about the system, two participants mentioned the clean interface and two mentioned the quick response. One participant mentioned it was easy to use; one liked the ease of logging in using a Google account, one liked the root cause, suggested solution and best practice in the recommendation and one liked the brown colour of the interface.

When asked what was confusing or difficult to use, one participant said there was nothing. The other four participants suggested improvements to the system. These include having a step-by-step solution provided in the recommendation, adding a percentage to the severity level, having a line under the recommendation stating "Suggested by AI. Please check before using." and adding a label to the theme button.

No participants mentioned any issues with functionality, which is consistent with all tasks being completed. The request for a label on the theme button relates to the effort to learn (item 10). Adding a percentage to the severity level and having an AI note are listed as future work (Section 5.2).

4.8 Discussion

This section discusses the results against the four objectives, the literature and the theory of Chapter 2. Table 4.12 summarises the evidence for each objective.

Table 4.12: Achievement of the research objectives

Objective	Evidence	Status
i. Severity classification	XGBoost model in the system; macro F1-score of 0.365; recall of 51.9% for Critical and 49.7% for Low, against 0% for the baseline.	Achieved, with low precision for the minority classes
ii. Solution recommendation	A three-part recommendation for every report; mean ratings from 4.14 to 4.34 out of 5.	Achieved; expert review remains
iii. Defect management efficiency	Dashboard, search, filters, status tracking and analytics; 100% task completion by five participants.	Achieved in a moderated test; time saving not measured
iv. Accuracy and performance	Macro-averaged metrics, majority-class baseline, 12 of 12 Selenium tests passed, mean SUS score of 69.0.	Achieved

4.8.1 Objective 1: Severity Classification

This achieves a reasonable level of detection for Critical and Low severity reports (around 50%), but at the cost of a low precision rate (around 1 in 8 Critical reports correctly predicted). This would be a useful tool for triage support (where a tester checks the output) but would not be suitable for automation.

There are several reasons for the low scores. These can be confirmed by undertaking an error analysis. Firstly, the model only uses the title of the report which only contains a few words. Secondly, the severity field in Bugzilla is inputted by reporters. Many reporters will leave this to the default value meaning noise in the labels. Thirdly, the six classes have been grouped into three meaning adjacent classes with similar wording will be grouped together. Finally, there is a large overlap in the vocabulary for the three classes. This is because the same words are used to describe a crash of different

importance. Previous studies have found similar issues where the task is simplified to two classes meaning they aren't directly comparable.

4.8.2 Objective 2: Solution Recommendation

All reports were given a 3-part recommendation and if the model service failed it gave a default message. The recommendations received an average score of above 4 for all 3 criteria (Table 4.8). The highest was Correctness at 4.34 and the lowest was Actionable at 4.14. This showed that the recommendations were seen as solving the problem described in the report and were technically correct. However, the recommendations were not as immediately usable. None of the recommendations scored below a 3. The average recommendations were remarkably similar across the severity levels, with Critical and Medium scoring 4.27 and Low scoring 4.20. The lowest average score was for report 1 at 3.87. Report 5 had the highest Actionable score at 5.0 but also the lowest Correctness score at 3.6. 5 friends of the author scored the recommendations. No expert checked the recommendations to see if they complied with the code of the application. This means that the scores are a preliminary indication of quality.

4.8.3 Objective 3: Defect Management Efficiency

The dashboard, filtering, tracking, and analytics made it possible for users to complete all the tasks from submission to completion. The functionality tests showed that the main function is dependable. There were three bugs identified during testing that have been fixed. The individual filtering allowed the testers data to be protected. All 5 participants completed all 4 tasks individually. The average SUS score was 69.0 which was one point higher than 68. The lowest scoring items were how easy it was to learn and how integrated the functions are. The first improvements to test would be having a brief guide on the first screen and shorter forms. The study did not time the triage with and without the tool, so the efficiency gain is supported by task completion and not by a time measurement.

4.8.4 Objective 4: Accuracy and Performance

The system was evaluated using different metrics including accuracy, macro metrics, baseline, confusion matrix, 12 functionality tests and usability survey. This is

important because when looking at just accuracy (55.5%) it does not show how well minority classes are being identified.

4.8.5 Comparison with the Literature and Theory

Table 4.13 places the result of IntelliBug beside the values reported in the reviewed studies.

Table 4.13: Reported results of related studies and IntelliBug

Study	Task	Metric	Value
Kukkar et al. [21]	Multi-class severity, five projects	Accuracy	96.34%
Mian and Alsaeedi [24]	Severity, Mozilla and Eclipse	Accuracy	91% and 89%
Arshad et al. [26]	Two classes, Eclipse and Mozilla	Accuracy	76.8%
Sarawan et al. [27]	Five levels, Mozilla	Accuracy	about 66%
Zhu et al. [28]	Five levels, cross-project	Macro F1	0.56
IntelliBug	Three classes, Eclipse, natural class distribution	Accuracy; macro F1	55.5%; 0.365

The scores of IntelliBug are lower than the published values, and three differences explain the deviation. First, the metric differs. Most studies report accuracy, and the majority-class baseline of this study reaches 88.9% accuracy with no useful prediction (Table 4.4). Among the values in Table 4.13, only the result of Zhu et al. [28] is a macro F1-score, and their best value of 0.56 is the closest reference for the 0.365 of IntelliBug. Second, the input differs. IntelliBug uses the title only, as a bag of words, while the recent studies use contextual embeddings from BERT or GPT-2 [25], [26], [28]. TF-IDF ignores word order and meaning, so two titles with the same words and a different impact receive the same vector. Third, the classes differ. This study keeps the natural share of 88.9% Medium reports in the test set and merges six Bugzilla values into three classes, so reports with similar wording fall on both sides of a class border.

The pattern of the errors agrees with the theory of Section 2.2.4. Cost-sensitive learning raises the cost of an error on a rare class, so the recall of the rare classes rises and their precision falls [8]. The model shows this trade exactly. The recall of Critical moved from 0% for the baseline to 51.9%, while 17,572 Medium reports were labelled Critical. The macro F1-score rose by 5.1 points, from 31.4% to 36.5%. The tuning

result agrees with the noisy-label finding of Lamkanfi et al. [2]. A larger vocabulary and two extra fields added no gain, which points to the labels and the short text, and not to the size of the model, as the limit.

The other two results follow the same reading. A mean SUS score of 69.0 sits at the benchmark of 68 [17] and in the OK to Good band of the adjective scale [16], and the wide confidence interval of five participants makes it an estimate. The high ratings of the recommendations agree with the repair results of Xia et al. [31], in which language models produced correct fixes from source code. IntelliBug gives the model no code, and the raters scored actionability lowest at 4.14, which is the expected effect of the missing context.

4.8.6 Significance of the Results

The results matter in three ways. For practice, a tool which finds half of the Critical reports gives a triage team a ranked starting list where a manual queue gives none, at the price of false alarms which a tester filters. For research, the study documents a baseline on a public dataset with the natural class distribution, macro-averaged metrics and a majority-class comparison, which later studies are able to reproduce and improve. For teaching, the project shows a complete AI-assisted quality assurance tool built and evaluated by one student in 12 weeks with open-source components.

4.8.7 Limitations

- The model was trained on titles, while users enter a title and a description.
- Data collected from a single open-source project. Results may vary for assorted products.
- Severity labels are subjective, and no human expert rated the reports again.
- The recommendation is generated without access to the source code of the application.
- The usability sample is small and made of students, and the system was tested on a local server.
- Five people (friends of the author) rated ten reports. Only one report was rated as Medium. Scores do not represent an overall result but give an initial indication.
- The time saved by the tool was not measured, so the efficiency gain rests on task completion.

4.9 Chapter Summary

The model achieved a macro F1 score of 0.365 and correctly identified approximately 50% of minority class reports. The majority baseline correctly identified 0% of reports. The system passed twelve automated tests and three issues were fixed. All tasks were completed by 5 participants and the average SUS score received was 69.0, 1 above the benchmark of 68. On average recommendations received a score between 4.14 and 4.34 out of 5 from raters. A low precision can be attributed to short text and noisy labels.

CHAPTER 5
CONCLUSION AND RECOMMENDATION

5.1 Conclusion

This project developed IntelliBug, a web system which predicts the severity of bug reports and recommends a solution. The four objectives were met as follows.

i. An XGBoost classifier with TF-IDF features and balanced weights was developed and integrated in a Flask application. It reached a macro F1 of 0.365 on 95,731 test reports.

ii. A Claude module that gives a root cause, recommendation and best practice has been created.

iii. A dashboard has been created to help manage the defect process. A report list, report page, status tracking, and analytics have been created. 12 tests have been automated.

iv. The system has been assessed using machine learning metrics, a baseline, functional tests, and a usability test with 5 people.

Three findings stand out. First, the balanced weights changed the behaviour of the model from ignoring the minority classes to finding about half of them. Recall reached 51.9% for Critical and 49.7% for Low reports, against 0% for a majority-class baseline. The price was low precision, 13.0% for Critical and 11.6% for Low. Second, the tuning experiments showed no gain from a larger vocabulary or from the product and component fields, so the short title and the noisy labels limit the model more than its settings do. Third, the complete tool worked for its users. All twelve automated tests passed, five participants completed every task alone, the mean SUS score was 69.0, and five raters scored the recommendations from 4.14 to 4.34 out of 5.

The study shows that it is possible to predict the severity of a bug report from short text. However, the precision of the minority classes is low. The benefit of the system is that it can predict, give advice and track the bug report. The final decision stays with the tester.

The project contributes a documented severity model on a public dataset, an integrated web system which joins prediction, repair advice and tracking, and an evaluation

which covers the model, the functions and the users. The results also carry limits. The model was trained on titles from one project, the raters and the participants were small groups of students, and the time saved by the tool was not measured. IntelliBug is therefore ready as a triage assistant under the control of a tester, and not as an automatic decision maker.

5.2 Recommendation for Future Work

The following improvements are recommended. Each one answers a limitation found in Chapter 4.

1. Train on the title and the description together, so training matches use, and compare the effect on macro F1. The description holds the steps and the error messages, which the title lacks.
2. Compare XGBoost with Logistic Regression, Naive Bayes, and transformer models such as BERT, using cross-validation. Contextual embeddings read word order, which TF-IDF ignores, and cross-validation gives a more stable estimate than one split.
3. Add a confidence score and a confirmation step, so low-confidence predictions are reviewed by a person. The model already computes the class probabilities, so the score only needs to be shown.
4. Rate the recommendations with professional testers and add the code context of the application to the prompt. Expert ratings will verify the correctness of the advice, and code context will make the advice specific.
5. Host the system on a cloud server. This will mean it can be accessed over HTTPS, using a production web server and managed database. Add a role system so that teams can be used.
6. Expand the data. This can be done by adding more bug reports from other projects, duplicate detection, and predicting priority.
7. Improve the display of the recommendation with a step-by-step resolution, a percentage beside the severity level, a line such as “Suggested by AI. Verify before use.” under the recommendation, and a label on the theme button, as the participants suggested.

8. Measure the efficiency gain directly by timing the triage of the same set of reports with and without IntelliBug.

REFERENCES

[1] Mozilla, "The Bugzilla guide: Bug fields." [Online]. Available: https://bugzilla.readthedocs.io/ (accessed Oct. 2026).

[2] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *Proc. 7th IEEE Working Conf. Mining Software Repositories (MSR)*, Cape Town, South Africa, 2010, pp. 1–10.

[3] T. Menzies and A. Marcus, "Automated severity assessment of software defect reports," in *Proc. IEEE Int. Conf. Software Maintenance (ICSM)*, Beijing, China, 2008, pp. 346–355.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[5] Anthropic, "Claude API documentation." [Online]. Available: https://docs.claude.com (accessed Oct. 2026).

[6] A. Lamkanfi, S. Demeyer, Q. D. Soetens, and T. Verdonck, "Comparing mining algorithms for predicting the severity of a reported bug," in *Proc. 15th European Conf. Software Maintenance and Reengineering (CSMR)*, Oldenburg, Germany, 2011, pp. 249–258.

[7] Y. Tian, D. Lo, and C. Sun, "Information retrieval based nearest neighbor classification for fine-grained bug severity prediction," in *Proc. 19th Working Conf. Reverse Engineering (WCRE)*, Kingston, ON, Canada, 2012, pp. 215–224.

[8] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[9] A. Lamkanfi, J. Pérez, and S. Demeyer, "The Eclipse and Mozilla defect tracking dataset: A genuine dataset for mining bug information," in *Proc. 10th Working Conf. Mining Software Repositories (MSR)*, San Francisco, CA, USA, 2013, pp. 203–206.

[10] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *J. Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

[11] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[12] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[13] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[14] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manage.*, vol. 45, no. 4, pp. 427–437, 2009.

[15] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.

[16] A. Bangor, P. T. Kortum, and J. T. Miller, "An empirical evaluation of the System Usability Scale," *Int. J. Human–Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008.

[17] J. Sauro and J. R. Lewis, *Quantifying the User Experience: Practical Statistics for User Research*, 2nd ed. Cambridge, MA, USA: Morgan Kaufmann, 2016.

[18] J. Nielsen and T. K. Landauer, "A mathematical model of the finding of usability problems," in *Proc. INTERCHI '93*, Amsterdam, The Netherlands, 1993, pp. 206–213.

[19] T. Zhang, J. Chen, G. Yang, B. Lee, and X. Luo, "Towards more accurate severity prediction and fixer recommendation of software bugs," *J. Syst. Softw.*, vol. 117, pp. 166–184, 2016, doi: 10.1016/j.jss.2016.02.034.

[20] W. Y. Ramay, Q. Umer, X. C. Yin, C. Zhu, and I. Illahi, "Deep neural network-based severity prediction of bug reports," *IEEE Access*, vol. 7, pp. 46846–46857, 2019, doi: 10.1109/ACCESS.2019.2909746.

[21] A. Kukkar, R. Mohana, A. Nayyar, J. Kim, B.-G. Kang, and N. Chilamkurti, "A novel deep-learning-based bug severity classification technique using convolutional neural networks and random forest with boosting," *Sensors*, vol. 19, no. 13, Art. no. 2964, Jul. 2019, doi: 10.3390/s19132964.

[22] Y. Tan, S. Xu, Z. Wang, T. Zhang, Z. Xu, and X. Luo, "Bug severity prediction using question-and-answer pairs from Stack Overflow," *J. Syst. Softw.*, vol. 165, Art. no. 110567, Jul. 2020, doi: 10.1016/j.jss.2020.110567.

[23] J. Uddin, R. Ghazali, M. M. Deris, R. Naseem, and H. Shah, "A survey on bug prioritization," *Artif. Intell. Rev.*, vol. 47, no. 2, pp. 145–180, 2017, doi: 10.1007/s10462-016-9478-6.

[24] T. S. Mian and A. Alsaeedi, "Software bug severity prediction using convolutional neural network and BiLSTM models," in *Advances in Intelligent Computing Techniques and Applications (IRICT 2023)*, F. Saeed, F. Mohammed, and Y. Fazea, Eds. Cham, Switzerland: Springer, 2024, pp. 1–12.

[25] A. Ali, Y. Xia, Q. Umer, and M. Osman, "BERT based severity prediction of bug reports for the maintenance of mobile applications," *J. Syst. Softw.*, vol. 208, Art. no. 111898, Feb. 2024, doi: 10.1016/j.jss.2023.111898.

[26] M. A. Arshad, A. Riaz, R. Fatima, and A. Yasin, "SevPredict: Exploring the potential of large language models in software maintenance," *AI*, vol. 5, no. 4, pp. 2739–2760, Dec. 2024, doi: 10.3390/ai5040132.

[27] K. Sarawan, J. Polpinij, G. Somprasertsri, and B. Luaphol, "Analyzing hybrid feature representations for improved multiclass bug severity classification," *Eng. Technol. Appl. Sci. Res.*, vol. 15, no. 4, pp. 24561–24569, Aug. 2025, doi: 10.48084/etasr.11090.

[28] L. Zhu, S. Wiangsamut, and J. Polpinij, "Integrating lightweight transformers for cross-project bug severity classification: An applied AI approach in software engineering," *Appl. Sci.*, vol. 16, no. 12, Art. no. 6026, Jun. 2026, doi: 10.3390/app16126026.

[29] J. Anvik, L. Hiew, and G. C. Murphy, "Who should fix this bug?" in *Proc. 28th Int. Conf. Software Engineering (ICSE)*, Shanghai, China, 2006, pp. 361–370.

[30] X. Hou et al., "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, Art. no. 220, Dec. 2024, doi: 10.1145/3695988.

[31] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, Melbourne, Australia, 2023, doi: 10.1109/ICSE48619.2023.00129.

[32] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, "What makes a good bug report?" in *Proc. 16th ACM SIGSOFT Int. Symp. Foundations of Software Engineering (FSE)*, Atlanta, GA, USA, 2008, pp. 308–318.

[33] P. Chapman *et al.*, "CRISP-DM 1.0: Step-by-step data mining guide," SPSS Inc., Chicago, IL, USA, 2000.

[34] K. Beck *et al.*, "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org/ (accessed Oct. 2026).

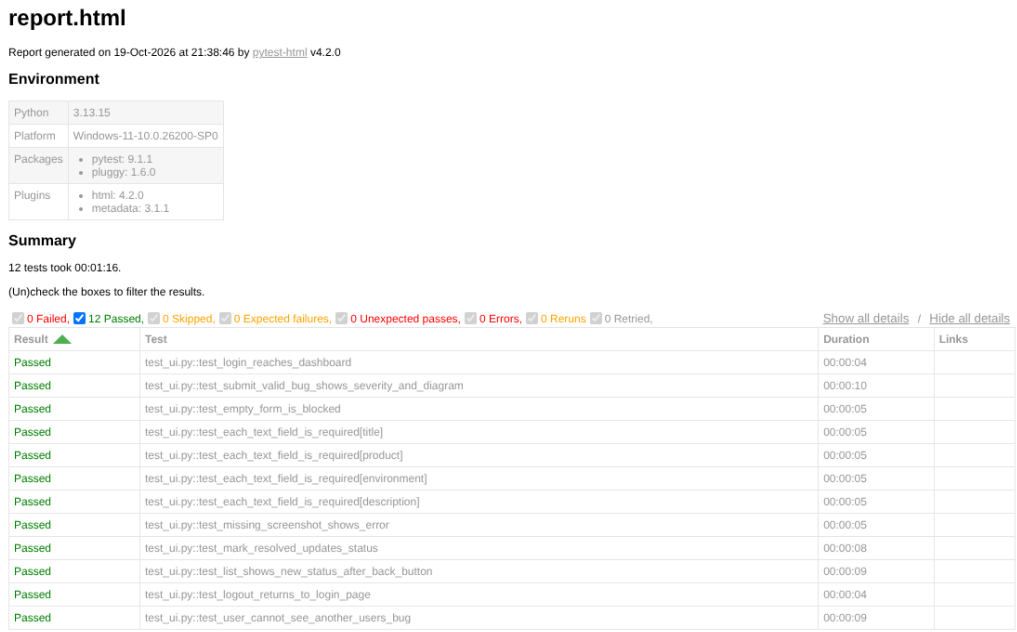
[35] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/ (accessed Oct. 2026).

[36] Selenium Project, "Selenium documentation." [Online]. Available: https://www.selenium.dev/documentation/ (accessed Oct. 2026).

APPENDICES

Appendix A. Selenium Test Suite (test_ui.py)

Figure A.1 shows the HTML report of the final run of the Selenium suite. All 12 test cases passed in 1 minute 16 seconds. The listing of the test file follows the figure.



report.html

Report generated on 19-Oct-2026 at 21:38:46 by pytest-html v4.2.0

Environment

Python	3.13.15
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1

Summary

12 tests took 00:01:16.

(Un)check the boxes to filter the results.

0 Failed, 12 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
Passed	test_ui.py::test_login_reaches_dashboard	00:00:04	
Passed	test_ui.py::test_submit_valid_bug_shows_severity_and_diagram	00:00:10	
Passed	test_ui.py::test_empty_form_is_blocked	00:00:05	
Passed	test_ui.py::test_each_text_field_is_required[title]	00:00:05	
Passed	test_ui.py::test_each_text_field_is_required[product]	00:00:05	
Passed	test_ui.py::test_each_text_field_is_required[environment]	00:00:05	
Passed	test_ui.py::test_each_text_field_is_required[description]	00:00:05	
Passed	test_ui.py::test_missing_screenshot_shows_error	00:00:05	
Passed	test_ui.py::test_mark_resolved_updates_status	00:00:08	
Passed	test_ui.py::test_list_shows_new_status_after_back_button	00:00:09	
Passed	test_ui.py::test_logout_returns_to_login_page	00:00:04	
Passed	test_ui.py::test_user_cannot_see_another_users_bug	00:00:09	

Figure A.1: Final run of the Selenium suite in the pytest-html report, 19 October 2026

```
"""IntelliBug UI tests: login, submission, required fields, status change, logout."""
import pytest
from pages.login_page import LoginPage
from pages.submit_bug_page import SubmitBugPage
from pages.bug_reports_page import BugReportsPage, BugDetailPage


def test_login_reaches_dashboard(driver, base_url):
    login_page = LoginPage(driver, base_url)
    login_page.login_as_demo()
    assert "IntelliBug" in driver.title


def test_submit_valid_bug_shows_severity_and_diagram(driver, base_url, test_image):
    LoginPage(driver, base_url).login_as_demo()

    submit_page = SubmitBugPage(driver, base_url)
    submit_page.open()
    submit_page.fill_form(
        title="Login button unresponsive on Safari",
        product="Customer Portal",
        environment="Safari 17, macOS",
        description="Expected: user reaches dashboard. Actual: nothing happens after
clicking Login.",
    )
    submit_page.attach_screenshot(test_image)
    submit_page.submit()

    submit_page.wait_for_result()
    assert submit_page.get_severity() in ("Critical", "Medium", "Low")
    assert submit_page.count_flow_steps() >= 1
```

```
# ---------------------------------------------------------------
# Stage 3: required fields, status change, logout
# ---------------------------------------------------------------
FIELDS = {
    "title": SubmitBugPage.TITLE,
    "product": SubmitBugPage.PRODUCT,
    "environment": SubmitBugPage.ENVIRONMENT,
    "description": SubmitBugPage.DESCRIPTION,
}
VALUES = {
    "title": "Checkout page freezes",
    "product": "Customer Portal",
    "environment": "Chrome 126, Windows 11",
    "description": "Expected: order completes. Actual: the page freezes.",
}


def test_empty_form_is_blocked(driver, base_url):
    LoginPage(driver, base_url).login_as_demo()
    submit_page = SubmitBugPage(driver, base_url)
    submit_page.open()
    submit_page.submit()
    assert submit_page.is_missing(SubmitBugPage.TITLE)
    assert not submit_page.is_result_visible()


@pytest.mark.parametrize("empty_field", list(FIELDS))
def test_each_text_field_is_required(driver, base_url, empty_field):
    LoginPage(driver, base_url).login_as_demo()
    submit_page = SubmitBugPage(driver, base_url)
    submit_page.open()
    for name, locator in FIELDS.items():
        if name != empty_field:
            driver.find_element(*locator).send_keys(VALUES[name])
    submit_page.submit()
    assert submit_page.is_missing(FIELDS[empty_field])
    assert not submit_page.is_result_visible()


def test_missing_screenshot_shows_error(driver, base_url):
    LoginPage(driver, base_url).login_as_demo()
    submit_page = SubmitBugPage(driver, base_url)
    submit_page.open()
    submit_page.fill_form(**VALUES)
    submit_page.submit()
    error = submit_page.wait_for_error()
    assert "attach a screenshot" in error.text
    assert not submit_page.is_result_visible()


def test_mark_resolved_updates_status(driver, base_url, new_bug):
    reports_page = BugReportsPage(driver, base_url)
    reports_page.open()
    reports_page.open_bug(new_bug)

    detail_page = BugDetailPage(driver)
    detail_page.mark_resolved()
    assert detail_page.wait_for_status("Resolved") == "Resolved"


def test_list_shows_new_status_after_back_button(driver, base_url, new_bug):
    reports_page = BugReportsPage(driver, base_url)
    reports_page.open()
    assert reports_page.row_status(new_bug) == "Open"
    reports_page.open_bug(new_bug)

    detail_page = BugDetailPage(driver)
    detail_page.mark_resolved()
    detail_page.wait_for_status("Resolved")

    driver.back()
    reports_page.wait_for_row_status(new_bug, "Resolved")


def test_logout_returns_to_login_page(driver, base_url):
```

```
    login_page = LoginPage(driver, base_url)
    login_page.login_as_demo()
    login_page.logout()
    assert "/login" in driver.current_url


def test_user_cannot_see_another_users_bug(driver, base_url, new_bug):
    # new_bug was reported by the demo user. Now sign in as someone else.
    reports_page = BugReportsPage(driver, base_url)
    reports_page.open()
    bug_url = reports_page.bug_link(new_bug)

    LoginPage(driver, base_url).login_as("anna")

    # 1. The bug is not in the other user's list.
    reports_page.open()
    assert not reports_page.has_bug(new_bug)

    # 2. Opening its address directly sends them back to their own list.
    driver.get(bug_url)
    assert driver.current_url.rstrip("/").endswith("/reports")
    assert new_bug not in driver.page_source
```

Appendix B. System Usability Scale Questionnaire

Answer scale for every item: 1 = Strongly disagree, 2, 3, 4, 5 = Strongly agree.

Table B.1: SUS statements adapted for IntelliBug

No.	Statement	Scoring
1	I think that I would like to use IntelliBug frequently.	Answer − 1
2	I found IntelliBug unnecessarily complex.	5 − Answer
3	I thought IntelliBug was easy to use.	Answer − 1
4	I think that I would need the support of a technical person to be able to use IntelliBug.	5 − Answer
5	I found the various functions in IntelliBug were well integrated.	Answer − 1
6	I thought there was too much inconsistency in IntelliBug.	5 − Answer
7	I would imagine that most people would learn to use IntelliBug very quickly.	Answer − 1
8	I found IntelliBug very cumbersome to use.	5 − Answer
9	I felt greatly confident using IntelliBug.	Answer − 1
10	I needed to learn a lot of things before I could get going with IntelliBug.	5 − Answer

Open questions:

1. What did you like most about IntelliBug?
2. What was confusing or hard to use?

Figures B.1 to B.10 show the responses to the ten statements, and Figures B.11 and B.12 show the answers to the two open questions. The charts come from the online questionnaire form.

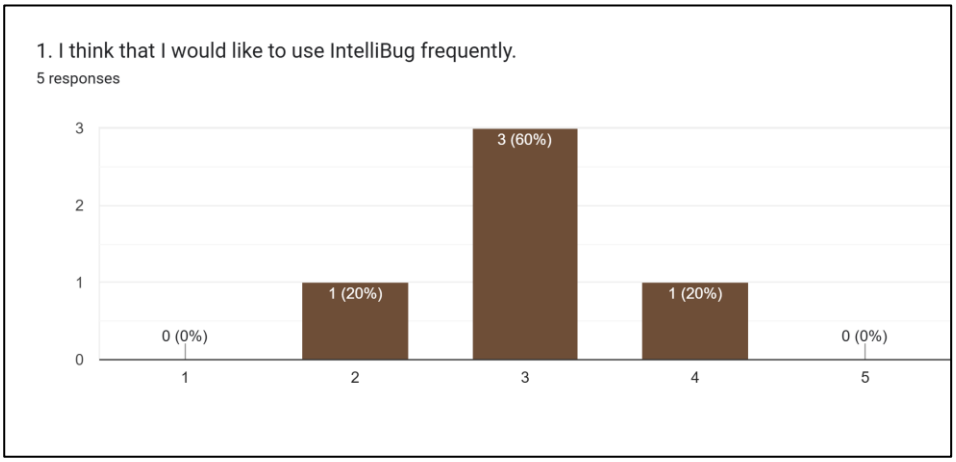


Figure B.1: Responses to SUS statement 1

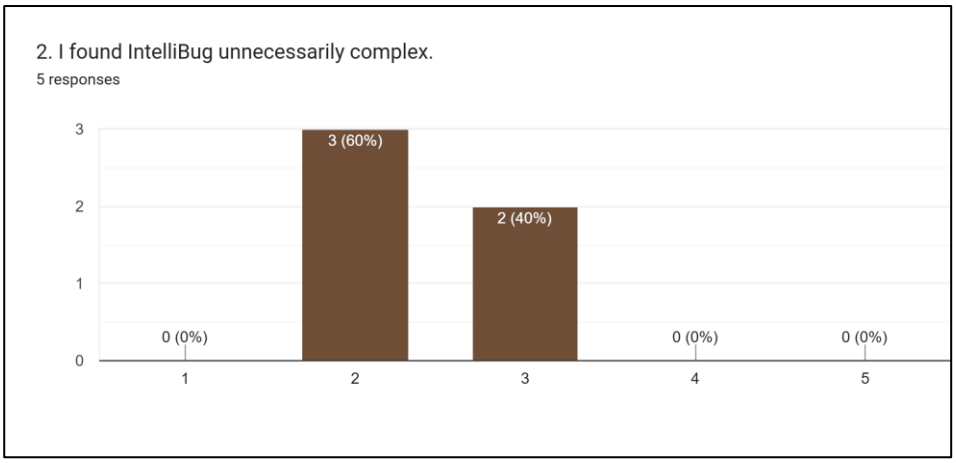


Figure B.2: Responses to SUS statement 2

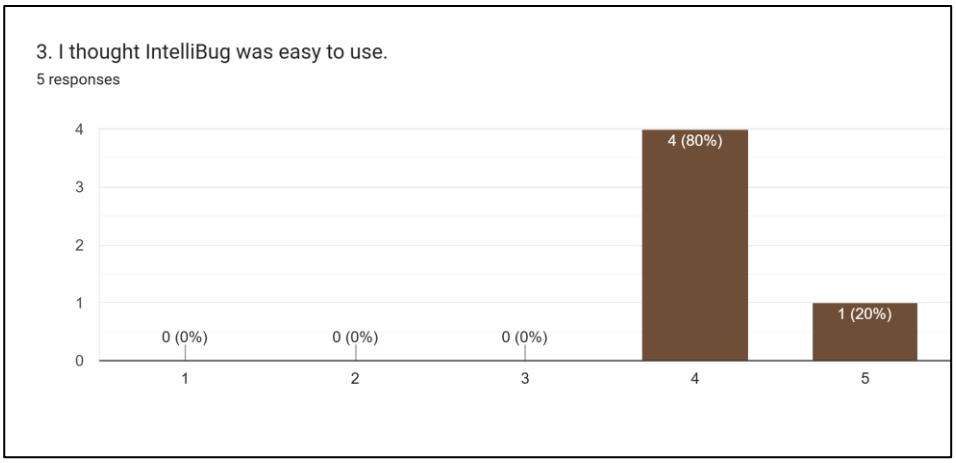


Figure B.3: Responses to SUS statement 3

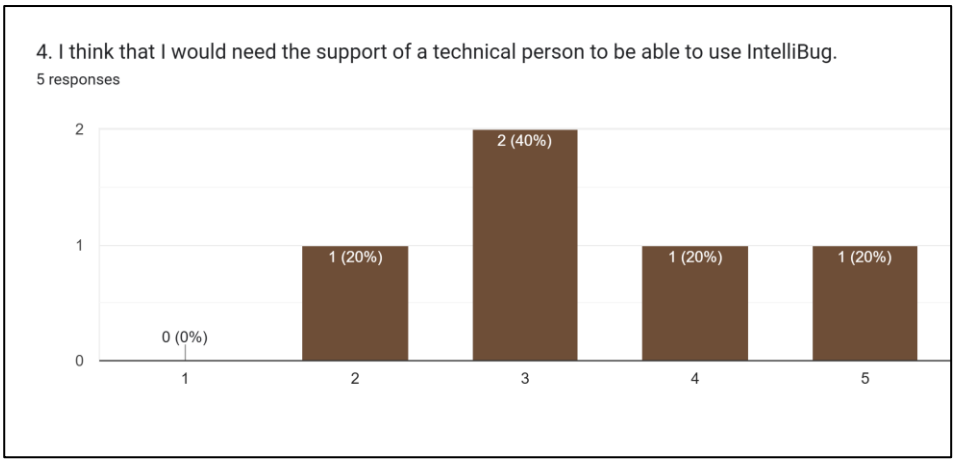


Figure B.4: Responses to SUS statement 4

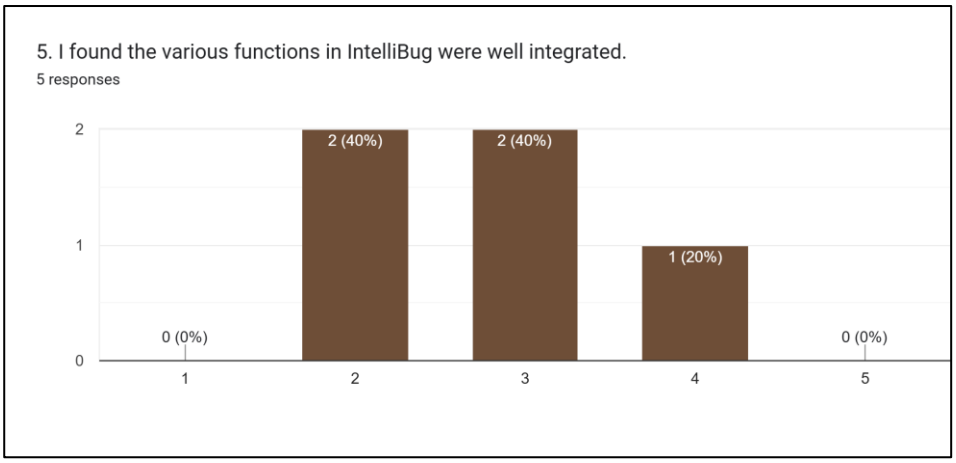


Figure B.5: Responses to SUS statement 5

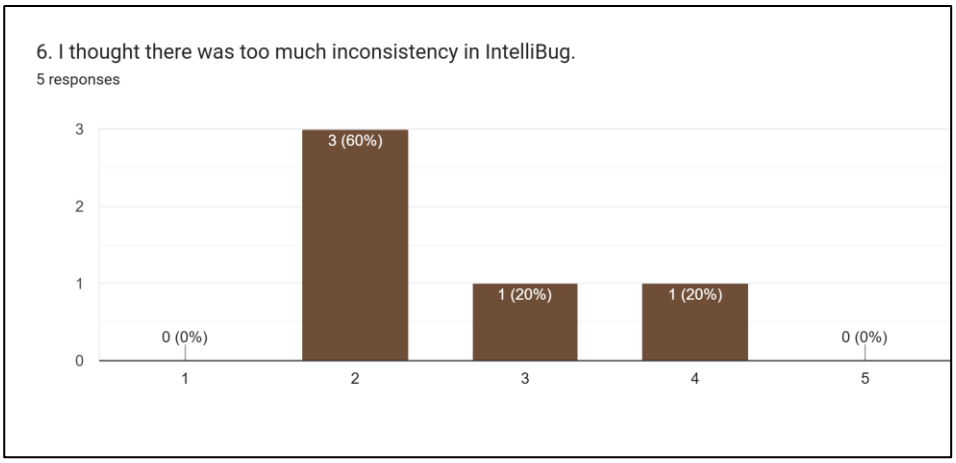


Figure B.6: Responses to SUS statement 6

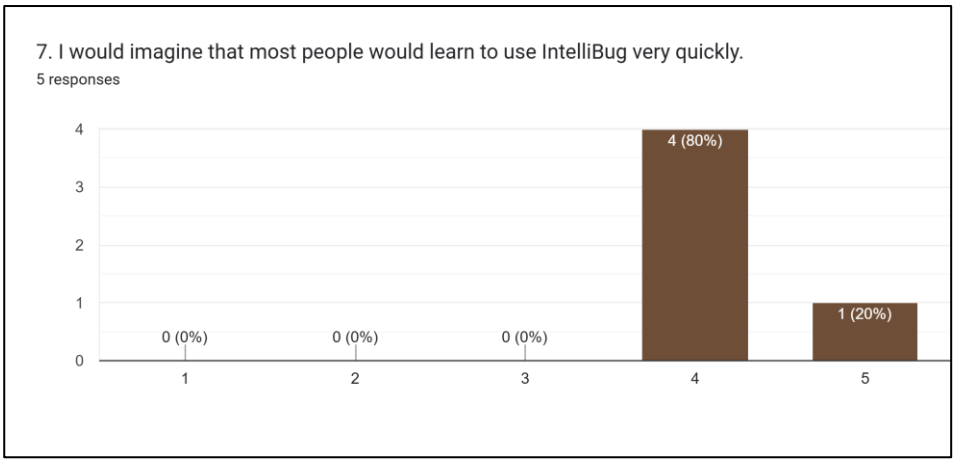


Figure B.7: Responses to SUS statement 7

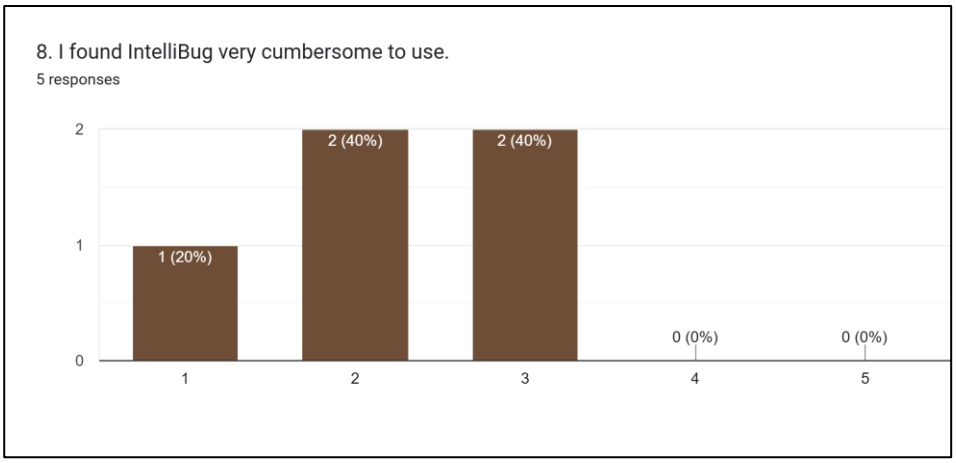


Figure B.8: Responses to SUS statement 8

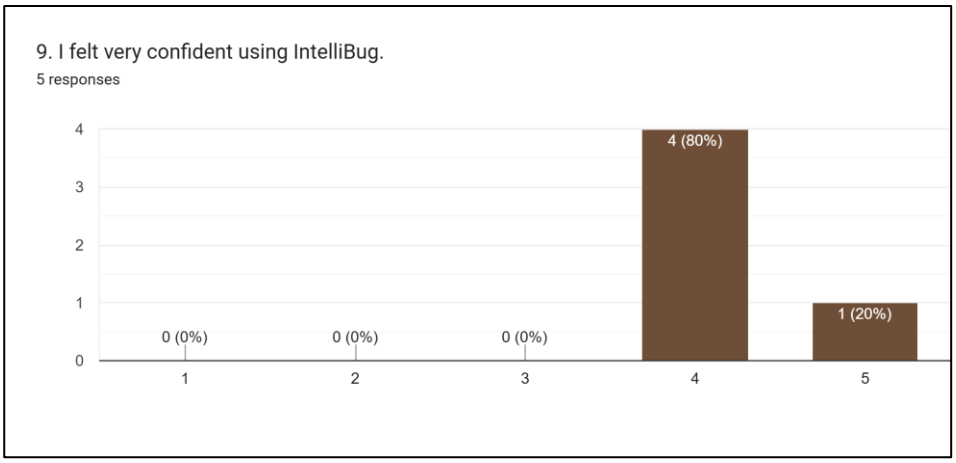


Figure B.9: Responses to SUS statement 9

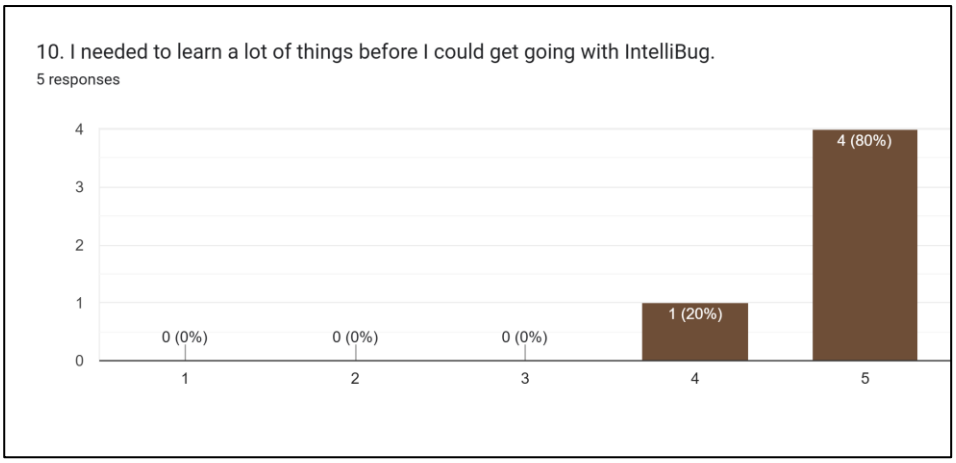


Figure B.10: Responses to SUS statement 10

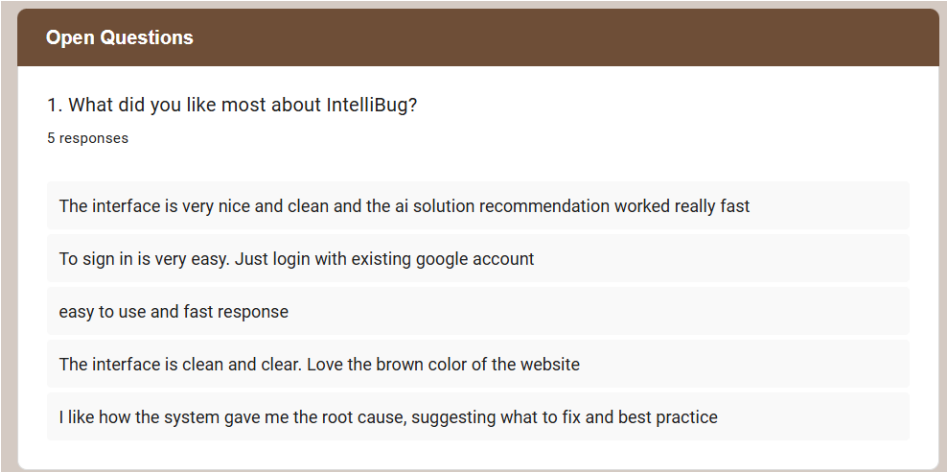


Figure B.11: Answers to open question 1, what did you like most about IntelliBug

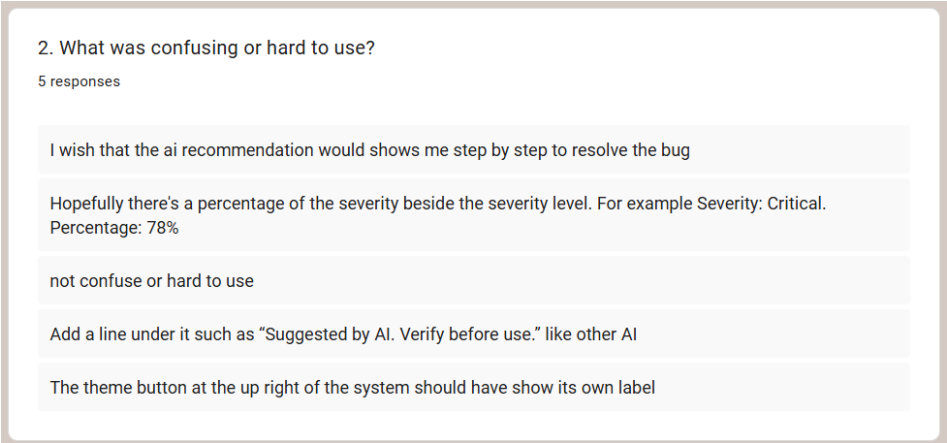


Figure B.12: Answers to open question 2, what was confusing or hard to use

Appendix C. Project Timeline (FYP II)

Table C.1 lists the eleven activities of FYP II, and Figure C.1 shows their schedule over twelve weeks.

Table C.1: FYP II activities

No.	Activity
1	Requirement refinement and project planning
2	Dataset collection and preparation
3	Data preprocessing (text cleaning, tokenisation, TF-IDF)
4	XGBoost model development and training
5	Model testing and evaluation
6	Web system development
7	Claude AI integration
8	QA dashboard development
9	System integration and functional testing
10	System refinement and bug fixing
11	Final documentation and presentation preparation

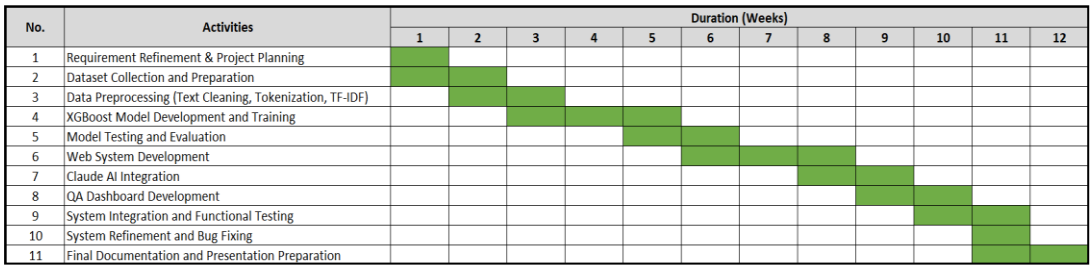


Figure C.1: Gantt chart of the FYP II activities over twelve weeks

Appendix D. Tuning Experiment Output

Figures D.1 and D.2 show the terminal output of the runs behind Table 4.5.

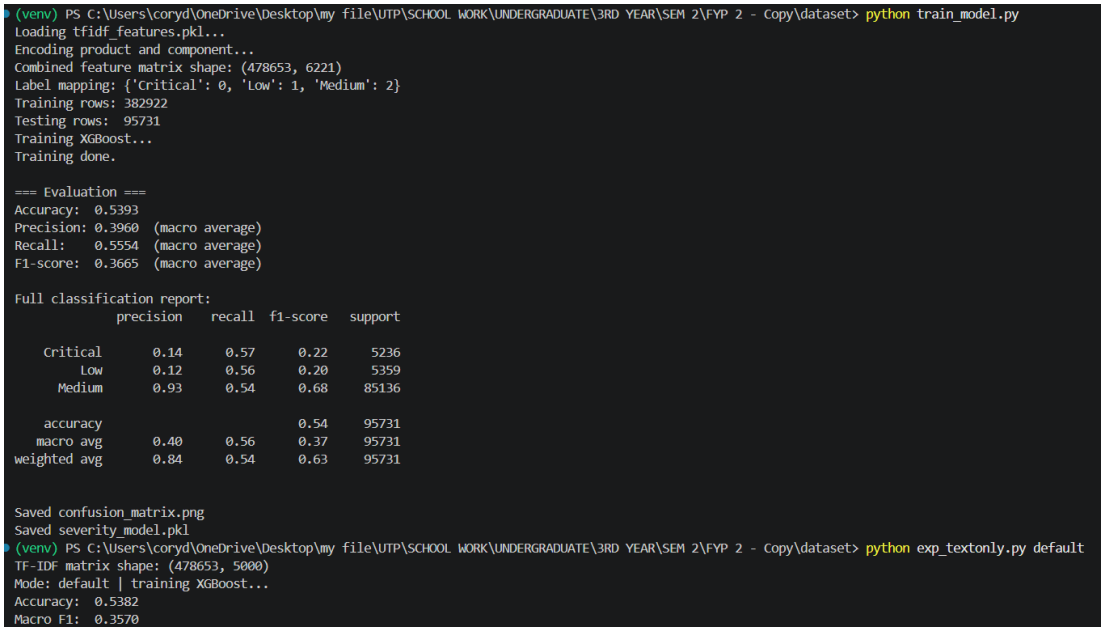


Figure D.1: Terminal output of the run with 5,000 TF-IDF terms

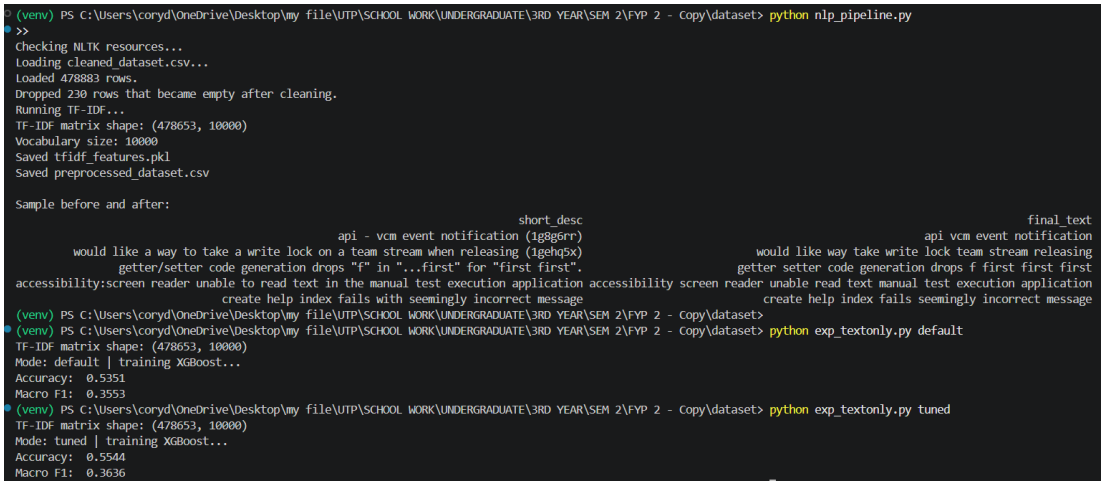


Figure D.2: Terminal output of the run with 10,000 TF-IDF terms

Appendix E. Source Code Repository

The full source code of IntelliBug, including the training scripts, the Flask application and the Selenium test suite, is available at https://github.com/leehana312/IntelliBug-FYP.